

Environnements de développement logiciel



Alexis ROYER - <https://www.linkedin.com/in/alexis-royer/>

2025-2026

Ce document constitue un cours d'ingénierie logicielle, plus particulièrement sur la mise en oeuvre des environnements de développement.

L'environnement de développement contribue à l'efficacité des activités, et par conséquent à la qualité d'un logiciel produit, d'où l'importance de maîtriser l'environnement de travail pour l'ensemble d'une équipe.

Source : <https://github.com/alxroyer/edu-sw-dev-env/>

Ce document est rédigé en format Markdown suivi sous git à dessein, pour démontrer le format Markdown et comment on peut l'utiliser pour la constitution de documentations techniques, ainsi que les capacités de suivi de version et de travail collaboratif apportées par git sur un format texte.

Cours mis à disposition sous licence Creative Common.

Voir le §6.1.

Sommaire :

- [1. Avant-propos](#)
 - [1.1. Types de logiciel](#)
 - [1.2. Technos et langages](#)
 - [1.3. Environnements de développement et qualité logicielle](#)
- [2. Développement](#)
 - [2.1. IDE - Integrated Development Environment](#)
 - [2.2. Debugging](#)

- 2.3. Bibliothèques de packages
 - 2.3.1. Fonctionnement
 - 2.3.2. Contraintes techniques
 - 2.3.3. Contraintes juridiques
 - 2.3.3.1. Licences commerciales
 - 2.3.3.2. Licences Open Source
 - 2.3.3.3. Licences hybrides
 - 2.3.4. Risques de sécurité
 - 2.3.5. Utilisation d'un miroir local
- 2.4. Build
 - 2.4.1. Compilateurs / linkers
 - 2.4.2. Bundlers Javascript
 - 2.4.3. Transpilation Typescript
 - 2.4.4. Gestionnaires de projets
 - 2.4.4.1. Makefile, les origines
 - 2.4.4.2. CMake
 - 2.4.4.3. Des configurations de mieux en mieux intégrées
- 2.5. Industrialisation
 - 2.5.1. Pratiques d'équipe
 - 2.5.2. Règles de codage
 - 2.5.3. Git
 - 2.5.4. Automatisations
 - 2.5.4.1. Scripting
 - 2.5.4.2. Chaîne CI - Continuous Integration
 - 2.5.5. Conteneurs Docker d'outillages
 - 2.5.6. Documentation
- 3. Exécution
 - 3.1. Exécution des applications PC
 - 3.2. Exécution des applications Web
 - 3.2.1. Exécution locale
 - 3.2.2. Exécution distante + développement en Remote SSH
 - 3.2.3. Conteneurs Dockers locaux
 - 3.2.4. Chaîne CD sur plateforme de test
 - 3.3. Exécution des applications mobile
 - 3.3.1. Exécution sur smartphone physique
 - 3.3.2. Emulateurs sur poste de développement
 - 3.3.3. Emulateurs dans le cloud
 - 3.3.4. Expo & Expo Go
 - 3.4. Exécution des logiciels embarqués
 - 3.4.1. Exécution sur cible hardware
 - 3.4.1.1. OS riche
 - 3.4.1.2. Bootloader
 - 3.4.1.3. Programmation mémoire
 - 3.4.2. Exécution dans un émulateur
 - 3.4.3. Exécution en mode simulé X86
 - 3.5. Stratégie de logs
- 4. Delivery / déploiement
 - 4.1. Enregistrements de livrables
 - 4.1.1. Répertoires partagés, GED
 - 4.1.2. Enregistrement sous git
 - 4.1.3. Dépôt de binaires
 - 4.1.4. Enregistrement dans les dépôts publics
 - 4.2. Applications PC
 - 4.2.1. Applications Windows
 - 4.2.2. Applications Linux

- 4.2.3. Applications Mac
- 4.3. Applications Web
 - 4.3.1. Chaîne CD - Continuous Delivery ou Continuous Deployment
 - 4.3.2. Infrastructure As Code IaC / GitOps
 - 4.3.3. Points de vigilance
 - 4.3.4. Scalabilité
 - 4.3.5. Pour en savoir plus
- 4.4. Applications mobiles
- 4.5. Logiciels embarqués
 - 4.5.1. Programmation initiale
 - 4.5.2. Mises à jour logicielles
- 5. Assurance qualité
 - 5.1. Versioning / Gestion de configuration
 - 5.1.1. SemVer
 - 5.1.2. SCM - git
 - 5.1.2.1. Git, les bases
 - 5.1.2.2. Investiguer avec git
 - 5.1.2.3. Stratégie git
 - 5.1.2.3.1. Format des commit logs
 - 5.1.2.3.2. Commit initial
 - 5.1.2.3.3. Merge v/s rebase
 - 5.1.2.3.4. Stratégie de branching
 - 5.1.2.3.5. Tags
 - 5.1.2.4. Plateformes git
 - 5.1.2.5. Git, mode expert
 - 5.1.3. Ticketing
 - 5.1.3.1. Fonctionnement général
 - 5.1.3.2. Intégration avec git
 - 5.1.3.3. Organisation du travail d'équipe
 - 5.1.4. Index de configuration
 - 5.1.5. PLM - Product Lifecycle Management
 - 5.1.6. Archivage
 - 5.2. Automatisations
 - 5.3. Documentation
 - 5.3.1. GED / répertoires partagés
 - 5.3.2. CMS / Wiki
 - 5.3.3. Formats texte + git
 - 5.3.3.1. Markdown
 - 5.3.3.2. RST / ReStructured Text
 - 5.3.3.3. PlantUML
 - 5.3.3.4. Mermaid
 - 5.3.3.5. Slides
 - 5.3.4. Documentation d'API
- 6. Annexes
 - 6.1. Licence
 - 6.2. Ressources

1. Avant-propos

1.1. Types de logiciel

Avant toute chose, il convient de considérer que la mise en oeuvre d'un environnement de développement est fortement dépendante du type de logiciel développé.

On distingue différents types de logiciels :

-  les applications PC (Windows, Mac, Linux),
-  les applications web (front et back),
-  les applications mobile (Android, iPhone),
-  les logiciels embarqués.

1.2. Technos et langages

Pour chacun des types de logiciels, des technologies différentes peuvent être mises en jeu (liste non exhaustive) :

-  Applications PC :
 - Langages :
 - C / C++
 - Java
 - Node.js (1) / Typescript (2)
 - Python
 - Go : exemple [esbuild](#) (6)
 - Rust (3)
 - Systèmes d'exploitation :
 - Windows
 - Mac
 - Linux
 - Frameworks :
 - Qt (4)
 - React Native (4)
-  Applications web :
 - Langages :
 - HTML
 - Côté *frontend* :
 - Javascript / Typescript (2)
 - Côté *backend* :
 - Node.js (1) / Typescript (2)
 - Java
 - PHP
 - Ruby
 - Python
 - C / C++
 - Go (6)
 - Frameworks :
 - Angular (Javascript)
 - Vue.js (Javascript)
 - React Native (Javascript) (4)
 - Symfony (PHP)
 - Ruby on Rails
-  Applications mobiles :
 - Architectures / systèmes d'exploitation :
 - Android
 - iOS
 - Langages :
 - Java, Kotlin (Android)

- Swift (iOS)
- Objective-C (iOS)
- C (Code natif Android & iOS)
- Frameworks :
 - React Native (4)
- Kits de développement :
 - SDK Android : Software Development Kit
 - NDK Android : Native Development Kit
-  Logiciels embarqués :
 - Langages :
 - C / C++
 - Go ? à confirmer (6)
 - Rust (3)
 - Systèmes d'exploitation :
 - Linux : couramment utilisé pour réaliser des systèmes embarqués. (5)
 - vxWorks : système d'exploitation dit *temps-réel*, propriétaire, développé par Wind River.
 - µC-OS : OS minimaliste, sans système de fichier, pile IP, ... par défaut.
 - Sans OS : simple boucle de réaction sur événement.
 - Frameworks / kits de développement :
 - buildroot

Notes :

- (1) Node.js : extension de Javascript, initialement restreint aux navigateurs, avec exécution par un moteur dédié.
- (2) Typescript : extension de la syntaxe Javascript pour permettre le typage.
- (3) Rust est un langage de programmation compilé, relativement nouveau, dont un des objectifs principaux est la sécurisation de la gestion de la mémoire. De fait, certains le présentent comme le successeur de C / C++.
- (4) Certains frameworks ont l'avantage d'être *cross-platform*, c'est à dire qu'ils permettent de réaliser un seul développement utilisable sur différents environnements (plateformes et/ou systèmes d'exploitation).
- (5) L'OS Linux, même dans une version embarquée, n'est pas considéré comme un OS *temps-réel*, c'est-à-dire qu'il ne garantit pas un temps d'exécution pour un traitement donné. L'occupation mémoire est plus importante que pour un OS minimaliste. En revanche, il a l'avantage de fournir un écosystème riche de services et d'outils qui peut faciliter les développements.
- (6) Go plebiscité pour ses capacités de gestion multi-thread, qui peut avoir son intrêt pour le développement d'applications avec des contraintes de perfs. Langage un peu spécifique, qui nécessite une phase d'apprentissage spécifique.

Le choix du langage et des technologies peut dépendre de plusieurs facteurs :

- en premier lieu, de la nature du logiciel développé,
- mais aussi des caractéristiques attendues : performances, empreinte mémoire, look & feel, ...
- de la culture d'entreprise : compétences disponibles en interne,
- de la stratégie d'entreprise : volonté d'investir sur telle ou telle technologie, ou de rationaliser les technologies utilisées, ...
- de la disponibilité des compétences sur le marché de l'emploi : capacité à staffer des équipes, assurer la maintenance dans le temps, ...
- ...

Quoi qu'il en soit, le langage et les technologies retenus conditionnent fortement l'environnement de développement à mettre en place.

i Vidéos pour le fun

Most popular coding language 2012 - 2024 : <https://www.youtube.com/watch?v=viPKPhOUcyU>

Most Popular Programming Languages 1955 - 2025 : https://www.youtube.com/watch?v=5yAbVkIML_M

1.3. Environnements de développement et qualité logicielle

La norme ISO 9126, ainsi que l'ISO 25010 qui la remplace aujourd'hui, identifient différentes dimensions pour la caractérisation de la qualité d'un logiciel. Entre autres (1) :

- Capacité fonctionnelle
- Facilité d'utilisation
- Fiabilité
- Performance
- Maintenabilité
- Portabilité

Une partie de ces dimensions reste essentiellement portée par le logiciel développé.

Toutefois, disposer des bons outils peut contribuer à certaines de ces dimensions, notamment :

- Fiabilité :
 - Mettre en oeuvre des outils d'analyse de code statique, permettant d'éviter des bugs avant même d'exécuter le code.
 - Exécuter régulièrement des tests de non-régression, pour détecter les problèmes au plus tôt.
- Maintenabilité :
 - Assurer un maximum de confort pour les développeurs, de sorte à faciliter le codage, le debugging... de par le choix des outils, pour leurs fonctionnalités, et sans oublier leurs performances ! Faire en sorte de limiter les freins aux différentes actions nécessaires à la réalisation d'un logiciel de qualité.
 - Faciliter notamment les refactorings, nécessaires à la maintenabilité.
 - Utiliser un outil de versioning (git).
 - Automatiser les vérifications de qualité (linters, indentation) pour s'assurer d'un certain niveau de lisibilité du code, donc un code plus facile à maintenir, à suivre en historique.
- Portabilité :
 - Exécution des tests de non-régression sur les différentes cibles.

i (1) Accès aux normes ISO

La difficulté des normes ISO est que celles-ci ne sont pas libres d'accès.

En effet, le site de référence de l'ISO ne fournit qu'une prévisualisation de la version en cours de la norme ISO 25010 : <https://www.iso.org/obp/ui/#iso:std:iso-iec:25010>

To view the full content, you will need to purchase the standard by clicking on the "Buy" button.

Toujours sur le site de référence, difficile de trouver une telle prévisualisation pour l'ISO 9126.

La liste simplifiée ci-dessus est tirée de la page Wikipédia https://fr.wikipedia.org/wiki/Qualit%C3%A9_logicielle

2. Développement

Avant de délivrer et exécuter le logiciel, commençons par mettre en oeuvre les éléments permettant de le construire.

2.1. IDE - Integrated Development Environment

Un des premiers outils qu'on est amené à aborder pour faire du développement logiciel est l'IDE.

L'usage d'un IDE présente de nombreux intérêts :

- **Edition de code :**

Basique, un simple notepad (voire **vi** pour les puristes !) pourrait faire le job.

- **Coloration syntaxique :**

Ca change déjà un peu la vie, dans la mesure où ça fluidifie la lecture du code.

- **Navigation :**

Capacité à aller directement sur le code d'une variable ou d'une fonction, ou dans le code Markdown d'un chapitre donné. Possibilité de gagner du temps, en évitant de scroller et de faire des CTRL+F dans tous les sens quand le code commence à prendre du volume.

- **Vérification syntaxique :**

Premier niveau de vérification du code, vis-à-vis de la syntaxe du langage utilisé, et vis-à-vis des références croisées des symboles définis dans le code (noms de variables, de classes, de fonctions, ...).

Les erreurs détectées sont généralement soulignées avec des vaguelettes rouges, et des marqueurs rouges pointent les fichiers avec erreurs.

- **Complétion :**

La complétion apporte un vrai plus, en évitant des fautes de frappe donc des erreurs chronophages au passage.

⚠ Attention : Complétion et fautes d'orthographe

Il existe un risque avec la complétion de répliquer des fautes d'orthographe ou des dyslexies sur les noms de variables ou de fonctions dans la totalité de code.

Le cas échéant, avoir le réflexe d'utiliser les fonction de refactoring pour corriger le tir.

- **Refactoring :**

On gagne encore plus du temps. On n'hésite plus à renommer une variable, une fonction, une classe, pour lui redonner un nom plus adapté, allant dans le sens de la maintenabilité.

- **Edition multi-lignes :**

Capacité mise à disposition par un certain nombre d'éditeurs de texte, à laquelle on s'habitue très vite dès lors qu'on commence à l'utiliser.

- **Debugging :**

Dès lors que c'est possible, permet d'obtenir plein d'informations utiles dans le cadre de résolution d'un bug. Possibilité également d'utiliser les fonctions de debugging pour tester des injections de fautes.

- **Ecosystème d'extensions :**

Les IDE les plus répandus viennent généralement avec un écosystème d'extensions riche permettant d'étendre les capacités de l'IDE.

Ces extensions permettent en premier lieu d'assurer le support des différentes technologies manipulées.

Elles permettent également d'améliorer l'efficacité au quotidien.

- **IA :**

Les IDE intègrent aujourd'hui des fonctions d'IA qui peuvent augmenter la productivité.

⚠ Attention : IA et confidentialité

A ce jour, l'IA fonctionne principalement par communication d'informations sur des moteurs s'exécutant sur Internet.

L'utilisation de l'IA dans les IDE peut donc être problématique si l'entreprise ne souhaite pas divulguer le code.

Bien se renseigner sur les usages possibles ou non.

Possible que l'entreprise dispose d'un contrat de confidentialité avec une IA donnée, voire héberge une solution d'IA *on-premises*, auquel cas on pourra configurer les IDE pour utiliser cette IA validée par l'entreprise.

On liste ci-après des IDE connus (liste non exhaustive) :

-  Applications PC :
 - Visual C++
 - Eclipse : Java, mais aussi C / C++ (version CDT)
 - VS Code : Node.js, Typescript, mais aussi C / C++ (extension)
 - JetBrains IntelliJ IDEA : Java
 - JetBrains PyCharm : Python
-  Applications web :
 - VS Code : HTML, Javascript, Typescript, PHP
 - PHPStorm
 - JetBrains IntelliJ IDEA : Java
-  Applications mobile :
 - Android Studio
 - JetBrains IntelliJ IDEA : Java, Kotlin
 - XCode : iOS
-  Logiciel embarqué :
 - VS Code : C / C++ (extension), Rust (extension)
 - Eclipse : C / C++ (version CDT)

Le tableau ci-après donne un aperçu historisé de l'apparition de certains des IDE précédemment cités :

Source : <https://chat.mistral.ai/>, sous réserve de confirmation des informations.

IDE	Langages	Description	Historique
Turbo Pascal	Pascal	L'un des premiers IDE modernes.	1983
Visual Studio	C, C++, C#	Développé par Microsoft.	1997
NetBeans	Java, PHP, C/C++	IDE polyvalent, version Open Source.	2000
Eclipse	Java, C++ (plugin CDT, 2004)	IDE open source, pour Java, en Java, extensible par plugins, réputation de lourdeur.	2001
IntelliJ IDEA	Java, ...	IDE populaire, développé par JetBrains, extensible par plugins.	2001
Xcode	C, Objective-C, Swift	IDE d'Apple pour le développement macOS et iOS.	2003
PyCharm	Python	Egalement développé par JetBrains.	2010
VS Code	HTML, CSS, JS/Typescript, Markdown, ...	Éditeur de code léger et extensible, développé en JS par Microsoft, écosystème riche de plugins.	2015



TP : [Configuration d'un IDE VS Code pour C/C++](#)



TP : Refactoring (TODO)

2.2. Debugging

i Environnements d'exécution

On parle de debugging dès à présent, car c'est une des fonctions majeures apportées par un IDE facilitant l'investigation et la résolution de bugs logiciels.

Cela entend qu'on sait exécuter notre logiciel au préalable (cf. §3). La notion reste toutefois abordable en considérant le cas simple des applications PC dans un premier temps.

Les debuggers (tels `gdb`) peuvent généralement s'utiliser en ligne de commande. Toutefois, cela reste extrêmement compliqué, et l'utilisation d'une interface graphique est quasiment indispensable pour déboguer efficacement.

Les IDE intègrent très bien les debuggers, et permettent ainsi de basculer très rapidement entre les modes édition de code et debugging / intégration.

Parmi les fonctions utiles d'un debugger, on note :

- la possibilité de positionner des points d'arrêt (*breakpoints*),
- l'exécution pas à pas, i.e. ligne à ligne,
- l'observation des valeurs de variables,
- rentrer dans l'exécution des fonctions (et en ressortir)
- l'affichage de la pile d'appels (*callstack*)
- la possibilité de définir un point d'arrêt sur modification de valeur : très utile pour investiguer des problèmes de débordement mémoire
- la capacité à modifier des valeurs de données : utile pour tester des injections de fautes
- la capacité pour certains langages interprétés (Python, JS) à pouvoir exécuter des routines à chaud
- ...



TP : Debugging C/C++ avec VS Code

Dans le cas du debugging d'un logiciel embarqué, la difficulté est de savoir synchroniser un IDE/debugger qui tourne sur une machine de développement, et le programme à déboguer qui tourne sur une cible autre. Pour ce faire, on peut utiliser des moyens spécifiques :

- `gdbserver` :
 - Possible pour des cibles tournant sous Linux uniquement.
 - Le programme à déboguer est lancé sur la cible avec `gdbserver`, et le debugger sur la machine de développement se connecte au `gdbserver` en tant que client.
 - J'ai déjà constaté personnellement que la connexion pouvait être instable. Pas de garantie notamment que les points d'arrêts soient bien pris en compte, notamment dans des cas d'applicatifs multi-threadés.
- Utilisation d'une sonde :
 - Matériel de sonde spécifique au micro-processeur de la cible.
 - Branchement sur un bus série dédié, directement sur le micro-processeur sur la cible. Nécessite que les signaux aient été routés sur la carte.



TP : Debugging C/C++ à distance avec gdbserver (TODO)

On note finalement que les investigations par debugging ont leurs limites :

- Pas de debugging possible *post-mortem*, i.e. après constat d'un problème après que l'exécution du logiciel soit terminée (cas typique d'une exception ou d'un crash constaté en prod) et tant qu'on n'a pas caractérisé les conditions de reproduction du problème.
- Il arrive fréquemment que le fait de poser des points d'arrêts provoque des effets de bords sur des timeouts qui arrivent à échéance, et donc des effets indésirables empêchant un debugging confortable.
- Dans le cas des logiciels embarqués, comme le logiciel s'exécute sur une cible autre, le debugging est par nature moins facile à mettre en oeuvre.
- Cas en Python où le débogueur exécute automatiquement des routines `__repr__()` pour obtenir une représentation des objets à afficher dans l'IHM de debug, ces exécutions surnuméraires pouvant provoquer des effets de bords indésirables selon les cas.

En raison de ces limitations, la bonne vieille technique du debugging "à la trace" ne doit pas être négligée (cf. §3.5).

2.3. Bibliothèques de packages

2.3.1. Fonctionnement

Aujourd'hui, on écrit rarement l'intégralité du code d'un logiciel. On repose généralement sur un écosystème riche de frameworks et de bibliothèques.

Ces écosystèmes de frameworks ou bibliothèques s'organisent généralement en deux parties :

- une *registry* en ligne : un site gérant une bibliothèque de bibliothèques.
 - Ces sites permettent de naviguer dans la bibliothèque de bibliothèques, faire des recherches, ...
 - Chaque bibliothèque est documentée : auteur, versions, dépendances, ...
 - On peut télécharger chacune des bibliothèques pour les installer *à la main*.

i Packages

Les frameworks ou bibliothèques pouvant être constitués de différents éléments (code source, licence, documentation, scripts utiles, ...), le téléchargement est facilité par le conditionnement dans une archive unique.

On nomme généralement cette archive un *package*.

Certaines technos affectent un nom spécifique pour ces packages, comme Rust qui nomme ses packages des *crates*.

- un programme s'exécutant en local, permettant de :
 - gérer un projet : build, debug, test, ... c'est l'équivalent de ce qu'on peut faire *à la main* avec un Makefile (cf. §2.4.4),
 - déclarer les packages qu'on souhaite utiliser au travers du projet géré par l'outil,
 - s'interfacer avec la *registry*,
 - résoudre les dépendances des bibliothèques.

Il existe différents systèmes de packages, généralement centrés sur une technologie donnée.

On liste ci-après des systèmes de packages usuels (liste non exhaustive) :

-  Applications PC :
 - JS : npm / yarn / pnpm (1)
 - Java : maven
 - Python : pip / venv
-  Applications web :
 - JS : npm / yarn / pnpm (1)
 - PHP : composer
 - Java : maven
-  Applications mobile :
 - Android : repo (2)
 - iOS : **swift package**
-  Logiciel embarqué :
 - C/Linux : buildroot (3)
 - Rust : Cargo

Le tableau ci-après donne un aperçu historisé de l'apparition de certains des systèmes de packages précédemment cités :

Source : <https://chat.mistral.ai/>, sous réserve de confirmation des informations.

Langage	Outil projet	Fichiers projet	Registry	Historique
Perl	cpan	cpanfile	https://metacpan.org/	1995
C/Linux	buildroot (3)	Fichiers <i>defconfig</i>	https://buildroot.org/	2001
Java	mvn V1 (2002), V2 (2008)	pom.xml	https://search.maven.org/ (2005)	2002

Langage	Outil projet	Fichiers projet	Registry	Historique
Python	pip, venv	requirements.txt, pyproject.toml, pylock.toml (4)	https://pypi.org/	2003
Ruby	gem	Gemfile	https://rubygems.org/	2004
JS	npm, yarn (2016), pnpm (2017)	package.json, package-lock.json	https://www.npmjs.com/	2010
PHP	composer	composer.json, composer.lock	https://packagist.org/	2012
Go	go get	go.mod, go.sum	https://pkg.go.dev/	2012
Rust	cargo	Cargo.toml, Cargo.lock	https://crates.io/	2014
Swift	swift package	Package.swift, Package.resolved	https://www.swift.org/package-manager/	2016

i Notes

- (1) yarn et pnpm constituent des améliorations de npm.
 - yarn, d'après <https://yarnpkg.com/> : "Safe, stable, reproducible projects"
 - pnpm, d'après <https://pnpm.io/> : "Save time. Save disk space. Supercharge your monorepos."

J'ai effectivement pu constater personnellement que npm pouvait avoir des limitations dans des cas particuliers.

C'était notamment le cas pour la gestion de différentes versions d'un même package (en l'occurrence React Native) pour les différentes targets dans un monorepo (cf. issue [npm#287](#)).

En raison de ces difficultés, j'étais passé sur yarn.

- (2) **repo** n'est pas vraiment un système de packages, mais plutôt une extension de git permettant de manipuler un grand nombre de dépôts dans l'espace projet.

Il n'y a pas vraiment de *registry* en ligne pour **repo**, mais simplement des dépôts git.

- (3) buildroot n'est pas vraiment centré sur un langage, mais sur des ensembles de bibliothèques et logiciels pouvant être embarqués dans l'image Linux constituée.

En un sens, cela ressemble plus à des systèmes de packages pour l'OS Linux, comme rpm (Red Hat Package Manager pour l'origine du nom) ou apt/dpkg (packages Debian .deb), à la différence que les packages ne sont pas téléchargés sous forme de binaires, mais de sources pour être cross-compilés.

On retrouvera classiquement pour Linux beaucoup de code C, mais pas uniquement.

- (4) Bien que la [PEP 518](#) (*Python Enhancement Proposal*) pour la définition des fichiers **pyproject.toml** date de 2016, la spécification des fichiers **pylock.toml** date de 2025 seulement (cf. <https://packaging.python.org/en/latest/specifications/pylock-toml/>).

Compte tenu du caractère récent de ces spécifications, l'usage des fichiers **requirements.txt** semble rester d'actualité.

👤 ✂ TP : [API REST en JS avec npm](#)

👤 ✂ TP : API REST en Python avec pip et venv (TODO)

👤 ✂ TP : API REST en PHP avec composer (TODO)

👤 ✂ TP : API REST en Go avec **go get** et **go build** (TODO)

Mémo :

- <https://go.dev/ref/mod#go-mod-init>

- <https://go.dev/doc/modules/managing-dependencies>
- <https://go.dev/doc/modules/gomod-ref>
- <https://go.dev/doc/tutorial/compile-install>

👨‍💻 TP : API REST en Rust avec cargo

👨‍💻 TP : Image Linux avec buildroot (TODO, penser à enregistrer le fichier *defconfig*)

Si le fait d'utiliser des librairies tierces permet de faciliter et d'accélérer les développements, cela ne va pas sans certains inconvénients.

2.3.2. Contraintes techniques

Comme on peut le voir, certains projets peuvent être amenés à tirer un volume conséquent de packages.

Dans ce genre de situation, de façon assez évidente, cela implique des contraintes sur l'espace disque utilisé.

Mais, avec un nombre important de packages utilisés, cela augmente également fortement le nombre d'accès disque, ce qui peut amener de vraies lenteurs sur les commandes de gestion des packages.

💡 Astuce : Privilégier un disque SSD

Il peut ainsi être intéressant de privilégier l'utilisation de disques SSD pour éviter des phénomènes de lenteur.

J'ai eu l'occasion de constater de vrais gains avec un disque SSD dans certains cas de figure :

- multiplicité des dépôts git Android tirés avec la commande `repo`,
- volume de packages tirés dans un monorepo JS avec React Native.

2.3.3. Contraintes juridiques

Toute librairie logicielle vient généralement avec une licence d'utilisation.

Les développeurs ont leur part de responsabilité vis-à-vis des licences tirées en fonction des choix de technologies opérés.

2.3.3.1. Licences commerciales

Il existe différents modes de facturation :

- au nombre de développeurs,
- par famille de produits,
- au runtime, i.e. par unité produite (souvent le plus coûteux),
- ...

Lorsque la licence est commerciale, penser à s'acquitter des droits à payer !

Tout oubli de règlement des licences commerciales peut se payer très cher devant les tribunaux, notamment lorsqu'on intègre les arriérés, plus probablement des pénalités à la clé !

2.3.3.2. Licences Open Source

Attention ! Open Source ne veut pas dire libre de droits.

Il existe différents types de licences Open Source plus ou moins permissives.

Le premier tableau ci-après donne une liste des principales licences dites permissives.

Ces licences permissives restent compatibles avec le développement d'un logiciel propriétaire, à la condition toutefois de se conformer aux termes de la licence.

Source : <https://chat.mistral.ai/>, sous réserve de confirmation des informations.

Licence	Identifiant SPDX (1)	Principales caractéristiques	Exemples
MIT (Massachusetts Institute of Technology)	MIT	Utilisation, modification et distribution libres, y compris dans des projets propriétaires. Seule obligation : conserver la notice de copyright et la décharge de responsabilité.	Ruby on Rails, jQuery
ISC (Internet Software Consortium)	ISC	Similaire à MIT, mais avec une formulation plus simple. Proposée par défaut dans l'écosystème JS.	npm
BSD (Berkeley Software Distribution)	BSD-3-Clause , BSD-2-Clause , 0BSD	Similaire à MIT, mais plus précise en termes de limitation de responsabilité du développeur vis-à-vis des usages faits du logiciel. Existe en différentes versions (nombre de clauses).	FreeBSD, NetBSD
Apache 2.0	Apache-2.0	Protection contre les brevets et clause de non-responsabilité. Compatible avec la GPLv3. Obligation de conserver les notices de licence et de ne pas utiliser les marques déposées.	Apache HTTP Server, Apache Kafka, une grande partie de code Android

Le second tableau suivant liste des licences dites *copyleft faible*.

Ce type de licences reste compatible avec le développement d'un logiciel propriétaire, mais impose généralement la reversion des modifications qui pourraient être apportées dans le code Open Source utilisé.

Source : <https://chat.mistral.ai/>, sous réserve de confirmation des informations.

Licence	Identifiant SPDX (1)	Principales caractéristiques	Exemples
LGPL (Lesser GPL)	LGPL-3.0-only , ...	Permet l'utilisation dans des logiciels propriétaires, mais les modifications de la bibliothèque LGPL doivent rester open source (i.e. obligation de publication des modifications).	GTK
MPL 2.0 (Mozilla Public License 2.0)	MPL-2.0	Copyleft faible par fichier.	Firefox, Thunderbird

Enfin, le troisième tableau liste des licences dites *copyleft fort* parmi les plus connues.

Ces licences imposent à un logiciel *dérivé* d'adopter lui-même la licence Open Source.

Ce qu'il faut entendre par logiciel *dérivé* c'est entre autres le fait de se *linker* avec le logiciel Open Source.

C'est pourquoi on qualifie ces licences de *contaminantes*.

De fait, une librairie avec ce type de licence est généralement incompatible avec le développement d'un logiciel propriétaire.

Source : <https://chat.mistral.ai/>, sous réserve de confirmation des informations.

Licence	Identifiant SPDX (1)	Principales caractéristiques	Exemples
GPL (GNU General Public License)	GPL-2.0-only , GPL-3.0-only , ...	Toute oeuvre dérivée doit être distribuée sous GPL également. La version en vigueur est la V3, mais on trouve également beaucoup de V2.	Linux, gcc

Licence	Identifiant SPDX (1)	Principales caractéristiques	Exemples
AGPL (<i>Affero GPL</i>)	AGPL-3.0-only , ...	Similaire à la GPL (licence contaminante), mais étend les obligations aux logiciels utilisés en réseau (SaaS).	itext
SSPL (<i>Server Side Public License</i>) (2)	SSPL-1.0	Licence créée par MongoDB, inspirée de AGPL.	MongoDB, ElasticSearch

i Notes

- (1) [SPDX](#) (*System Package Data Exchange*), est un standard ouvert permettant de spécifier la décomposition d'un logiciel, via une SBOM (*Software Bill of Materials*).

Ce standard propose des codification des licences logicielles (cf. <https://spdx.org/licenses/>).

- (2) Controverse : SSPL n'est pas reconnue comme une licence Open Source à part entière par l'[OSI](#) (*Open Source Initiative*). Cf. <https://opensource.org/blog/the-sspl-is-not-an-open-source-license>.

💡 **SPDX & conformité juridique**

Des outils associés au standard SPDX permettent de vérifier la conformité juridique aux licences embarquées.

Cf. <https://spdx.dev/use/spdx-tools/>.

i **CC - Licences Creative Commons**

Proche des licences Open Source, il existe également les licences Creative Commons.

Ce type de licence est plutôt dédié à la protection d'oeuvres créatives numériques, littéraires ou artistiques.

Cf. <https://creativecommons.org/>.

Note : Ce cours est publié sous licence CC BY-NC-SA. Cf. [LICENSE.txt](#).

2.3.3.3. Licences hybrides

Il arrive régulièrement que des projets Open Source adoptent un *business model* hybride :

- licence commerciale pour les entreprises,
- licence Open Source pour les utilisations personnelles, éducatives ou Open Source.

Exemples :

- Qt : licence [GPL](#) ou [LGPL](#) ou commerciale.
- ElasticSearch : licence [SSPL](#) ou [AGPLv3](#) ou commerciale.
- MongoDB : licence [SSPL](#) ou commerciale.
- itext : licence [AGPL](#) ou commerciale.

2.3.4. Risques de sécurité

Embarquer des librairies tiers signifie également embarquer les bugs que celles-ci contiennent, et pire encore leurs failles de sécurité !

C'est pourquoi il convient de mettre en place une MCS (Maintien en Conditions de Sécurité), c'est-à-dire une veille sur les alertes de sécurité et failles publiées.

Sources d'information pour la mise en oeuvre d'une veille MCS:

- CVE (*Common Vulnerabilities and Exposures*) :
 - Depuis 1999 (première CVE : [CVE-1999-0001](#)).

- Base de failles de sécurité connues pour les différents logiciels référencés.
- Plus de 300 000 CVE en base début février 2025.
- Page de recherche : <https://www.cve.org/CVERecord/SearchResults> (redirection depuis <https://cve.mitre.org/>).
- CERT-FR (French Computer Emergency Response Time) de l'ANSSI (Agence Nationale de la Sécurité des Systèmes d'Information) :
 - <https://www.cert.ssi.gouv.fr/>

Pour mener les analyses d'impact liées à l'occurrence d'une alerte de sécurité, il est important de se baser sur les fichiers `package-lock.json` ou équivalent, de sorte à vérifier la version des librairies effectivement résolue par le gestionnaire de packages.

! Retex Sha1-Hulud 2.0 (fin novembre 2025)

Fin novembre 2025, le CERT-FR publie l'actualité [CERTFR-2025-ACT-051](#) faisant état d'une "attaque par la chaîne d'approvisionnement de plusieurs paquets NPM".

Sujet également documenté sur le site de Microsoft : <https://www.microsoft.com/en-us/security/blog/2025/12/09/shai-hulud-2-0-guidance-for-detecting-investigating-and-defending-against-the-supply-chain-attack/>.

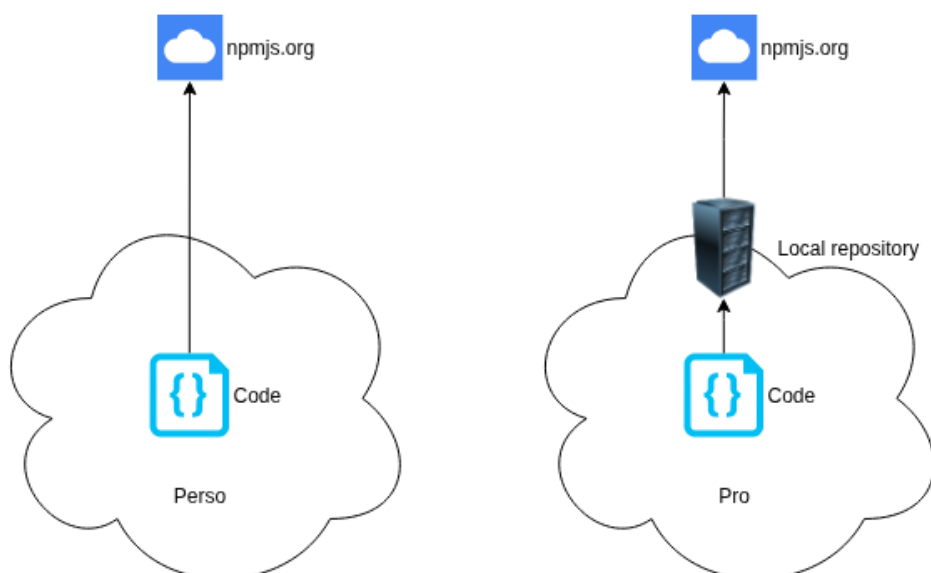
Pour faciliter les analyses d'impact, on a pu se baser sur un outil disponible sur GitHub : <https://github.com/gensecaiHQ/Shai-Hulud-2.0-Detector>.

En observant la [liste des fichiers supportés](#) par Shai-Hulud 2.0 Detector, on peut voir que cet outil considère les fichiers suivant pour mener l'analyse :

- `package.json`,
- `package-lock.json`,
- `yarn.lock`,
- `npm-shrinkwrap.json`,
- `pnpm-lock.yaml`.

2.3.5. Utilisation d'un miroir local

Les grandes entreprises déploient généralement un serveur intermédiaire entre les registries officielles sur Internet et le SI de l'entreprise.



Ces outils, présentés au §4.1.3 permettent d'enregistrer et mettre à disposition des fichiers binaires volumineux.

Ces outils peuvent également être configurés en relais de serveurs de packages.

Ce type de déploiement peut présenter les avantages suivants :

- accélération des téléchargements de packages, par la constitution d'un cache local, et l'apport de résilience en cas de perturbations réseaux avec Internet ;
- indépendance pour la maintenabilité à long terme : il peut arriver que certains packages disparaissent des registries, et on peut également anticiper que les registries risquent d'être arrêtées à l'avenir ;
- possibilité de renforts de la sécurité, avec mise en oeuvre de contrôles d'anti-virus sur les packages téléchargés depuis les registries ;
- capacités d'inventaires logiciels (SBOM), pouvant constituer une réponse aux questions juridiques et de sécurité évoquées précédemment.

2.4. Build

Après avoir choisi un langage pour le développement, ainsi qu'un IDE, après avoir sélectionné des frameworks et des librairies, il est l'heure de *builder* notre application, c'est-à-dire produire le livrable exécutable.

Selon le langage utilisé, on va considérer différents types de *builds*.

Le tableau ci-après propose une vue d'ensemble des modes de *build* pour chaque technologie, et des outils projet associés permettant de réaliser ces *builds* :

Techno	Type de build	Commande / outil de build	Outil projet	Fichiers projet
C / C++	Compilation (2)	gcc, g++	make, CMake	Makefile, CMakeLists.txt
Java	Compilation (3)	javac, jar	Maven	pom.xml
Javascript	Bundling	Webpack, Metro, esbuild, Vite	npm, yarn, pnpm	package.json, webpack.config.js, metro.config.js, vite.config.js
Typescript	Transpilation	tsc	tsc	tsconfig.json
PHP (1)	-	-	-	-
Python (1)	-	-	-	-
Rust	Compilation (2)	rustc	cargo	Cargo.toml
Go	Compilation (2)	go build	go build	go.mod

i Notes

- (1) Les langages interprétés n'ont pas de build en général, puisque par nature ils sont directement exécutables. A l'exception de Javascript qui embarque une notion de *bundling* qu'on va détailler après.

Les autres notes sont détaillées dans les chapitres correspondants, à suivre.

2.4.1. Compilateurs / linkers

Les langages dits compilés ne peuvent généralement être exécutés directement, car ils requièrent une étape de transformation du code source en code machine.

i Notes

- (2) En C / C++, Rust, Go, le binaire final est du code machine dépendant de l'architecture processeur sur lequel il va s'exécuter (ARM, X86, ... Big/Little Endian, 32 / 64 bits) ainsi que du système d'exploitation (Windows, Linux, ... cf. <https://stackoverflow.com/questions/48235579/why-do-we-need-to-compile-for-different-platforms-e-g-windows-linux#48236231>).
- (3) En Java, le *bytecode* généré est prévu pour être portable, car interprété par une JVM (*Java Virtual Machine*).

En règle générale, chaque fichier source est d'abord compilé unitairement, pour constituer un fichier intermédiaire de code machine :

- C / C++ : fichiers objets `.o`.
- Java : fichiers `.class`.
- Rust : (*sans objet*)

L'étape de build étant complètement intégrée par Cargo (cf. §2.4.4.3), les développeurs ne voient pas passer les fichiers objets générés de manière intermédiaire.

- Go : (*sans objet*), à confirmer

N'ayant pas d'expérience personnelle en Go, je suppose que, comme en Rust, les développeurs ne voient pas les fichiers objets intermédiaires.

Ensuite ces différents fichiers objets sont généralement *liés* ensemble pour constituer l'exécutable final.

i Java Archive

En java, on ne parle pas vraiment de link, mais d'archive `.jar` (*Java Archive*).

2.4.2. Bundlers Javascript

Les langages interprétés, par nature, ne nécessitent pas d'étape de *build*. Javascript introduit toutefois une notion de *bundling* adaptée au contexte du Web.

En effet, le bundling Javascript permet d'assembler plusieurs fichiers source en un seul, ce qui permet de développer de façon confortable dans plusieurs fichiers (et non dans un seul fichier énorme), mais de livrer un fichier Javascript unique, plus facile à télécharger par les navigateurs.

Le bundling permet d'assembler des sources Javascript, mais également d'embarquer des ressources telles que des styles CSS et des images.

Une opération de bundling précise également la version ECMAScript souhaitée pour le Javascript produit. Cette *transpilation* permet d'assurer une compatibilité maximale avec les différents navigateurs en sortie (ES5, voir tableau ci-après), tout en gardant le confort des dernières versions du standard en développement.

! Standards ECMAScript

L'[Ecma International](https://ecma-international.org/) est une [association d'acteurs industriels](#) qui définit et publie différents standards, dont le standard ECMAScript pour le langage Javascript : <https://ecma-international.org/technical-committees/tc39/>.

Le tableau ci-après recense quelques versions majeures du standard :

Source : <https://chat.mistral.ai/>, sous réserve de confirmation des informations.

Nom	Date	Fonctionnalités majeures	Compatibilité avec les navigateurs	Utilisation en développement
ECMAScript 1 (ES1)	1997	Première standardisation.	Très limitée, support historique.	Obsolète.
...				
ECMAScript 5 (ES5)	2009	Mode strict, JSON natif , méthodes pour les tableaux (map, filter, reduce), propriétés getters/setters.	Excellente, support universel.	Version la plus compatible , encore utilisée pour le support des anciens navigateurs.

Nom	Date	Fonctionnalités majeures	Compatibilité avec les navigateurs	Utilisation en développement
ECMAScript 6 (ES6/ES2015)	2015	Classes, modules, promesses, <code>let/const</code>, <code>arrow functions</code> , littéraux de gabarits, déstructuration, paramètres par défaut, opérateur de repos/spread.	Bonne, transpilation souvent nécessaire.	Révolution majeure , très utilisée en développement moderne.
ECMAScript 2016 (ES2016)	2016	Opérateur d'exponentiation (<code>**</code>) , méthode <code>Array.prototype.includes</code> , amélioration des fonctions asynchrones (<code>async/await</code>).	Bonne, transpilation parfois nécessaire.	Utilisée pour les fonctionnalités asynchrones avancées.
...				
ECMAScript 2020 (ES2020)	2020	Opérateur de coalescence nulle (<code>??</code>), opérateur optionnel d'enchaînement (<code>?.</code>), <code>import()</code> dynamique, <code>BigInt</code> , <code>Promise.allSettled</code> , <code>String.matchAll</code> .	Très bonne, support large.	Utilisée pour une gestion plus robuste des valeurs nulles et des promesses, ainsi que pour les grands entiers.
...				
ECMAScript 2025 (ES2025)	2025	Records & Tuples (types de données immutables), amélioration des décorateurs, nouvelles méthodes pour les tableaux et objets.	Partielle, en cours d'adoption.	Dernière version publiée.
ESNext	-	-	-	Disponible pour tester les nouvelles fonctionnalités. A réserver pour des usages expérimentaux.

A noter qu'à compter de 2015, une version sort tous les ans, au mois de juin. Aussi, le réalignement en développement sur la toute dernière version n'est pas forcément indispensable. A juger en fonction des fonctionnalités apportées.

Le bundling embarque généralement une opération dite *minify* qui réduit la taille du code Javascript généré, pour en accélérer le téléchargement sur Internet. Pour ce faire, cette étape :

- supprime les espaces et indentations, utiles pour la maintenance du logiciel dans les fichiers source d'origine, mais pas dans la production livrable en exécution,
- réduit la taille des noms de variables locales.

💡 uglify

Le bundling peut également embarquer une opération dite *uglify* constituant une obfuscation de premier niveau, pour éviter le rétro-engineering du code livré pour exécution.

⚠️ Javascript : un écosystème en constante évolution

Javascript est un écosystème riche, mais en constante évolution.

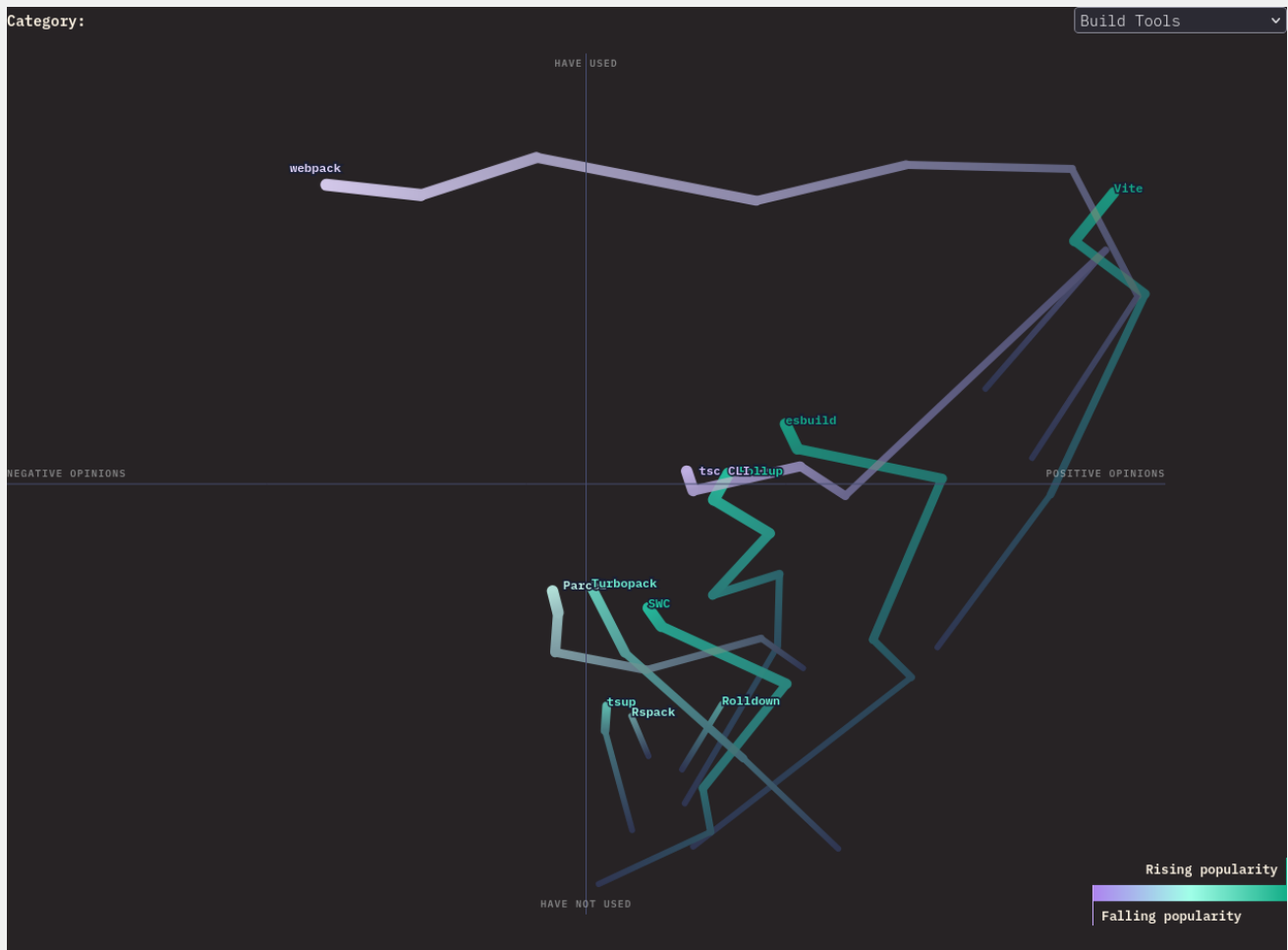
L'angle mort de cette vivacité est une cohérence d'ensemble pas toujours assurée.

Il en résulte souvent beaucoup de temps perdu dans la configuration des outils en raison de problèmes de compatibilité, notamment entre les différentes normes ECMAScript ou formats de modules (passage ES5 / ES6 souvent compliqué).

💡 Visualisation de la vivacité de l'écosystème Javascript

Le site <https://stateofjs.com/en-US> donne des illustrations graphiques très intéressantes sur l'évolution des différentes technologies dans le monde Javascript.

Chercher notamment les graphiques *Changes Over Time*.



Source : https://2025.stateofjs.com/en-US/libraries/#tools_arrows

On voit graphiquement la perte de popularité de Webpack, bien que bénéficiant toujours d'une forte notoriété, alors que esbuild et Vite gagnent fortement en notoriété et en popularité sur l'ensemble, avec toutefois un petit retrait de popularité pour esbuild dernièrement.

Autre site pouvant permettre de comparer différentes alternatives de solutions JS (mais pas que) : <https://stackshare.io/stackups/trending>

⚠ Des vitesses d'exécution des bundlers variables, impactantes pour la productivité

L'écosystème Javascript est un écosystème très riche, proposant de nombreux bundlers, avec des performances très variables.

D'expérience personnelle, certains bundlers tels Webpack et Metro sont extrêmement lents, et pénalisent la productivité des développements.

Le site <https://esbuild.github.io/> présente une animation amusante, illustrant les différences de performances qu'on peut observer entre Webpack et esbuild notamment. Bien que cette animation serve la promotion de la solution esbuild elle-même, je peux personnellement attester d'un certain réalisme de cette animation, de par les gains de performances ressentis en migrant de Metro à esbuild (développement React Native).

Or cette différence de performances de l'outillage au quotidien peut faire une vraie différence sur la productivité des activités de développement. Faire l'effort de changer de bundler peut s'avérer payant à moyen voire à court terme.

🧑🏻‍💻 TP : Bundling d'une application React Native avec Vite (TODO)

Mémo :

- <https://github.com/codepilots/ReactNativeVite>

2.4.3. Transpilation Typescript

Typescript constituant une extension de Javascript avec l'adjonction d'informations de typage, la transformation de code Typescript en Javascript ne constitue pas réellement une *compilation* mais une *transpilation*. En effet le code produit reste un langage de programmation (et non du code machine), qui plus est du code Javascript fortement similaire au code source d'origine.

Une transpilation Typescript peut permettre la production d'un code Javascript unique à partir de plusieurs fichiers Typescript en entrée. En ce sens, une transpilation Typescript peut opérer comme un bundler.

Mais la transpilation Typescript est plus généralement intégrée dans une opération de bundling prenant des fichiers Typescript en entrée à transpiler, puis à bundler ensuite.

2.4.4. Gestionnaires de projets

L'opération de build peut requérir plusieurs étapes, et donc nécessiter un gestionnaire de projet permettant de réaliser cela.

2.4.4.1. Makefile, les origines

En C / C++, il fallait souvent en faire beaucoup "à la main" avec les fichiers Makefile. Dès lors, on était régulièrement amené à déboguer ces fichiers.

Langage	Outil projet	Fichier projet	Documentation
C / C++	make	Makefile	https://www.gnu.org/software/make/manual/html_node/index.html

Comme beaucoup de projets C / C++ actifs utilisent encore des Makefile, il est intéressant d'en présenter rapidement les principes généraux :

- Les Makefiles sont globalement des collections de [règles](#) :

```
target: dep1 dep2
    command1 --options arg1 arg2
    command2 --options arg1 arg2
```

- La première règle dans le fichier constitue la règle par défaut.
- Une règle permet de résoudre une *target*.
 - La target est classiquement un fichier à construire.
 - Mais il peut aussi d'agir d'une target [phony](#) i.e. une règle nommée, sans correspondance avec un fichier réel, (typiquement une règle [clean](#)).
- Une règle définit des dépendances (*dep1*, *dep2* dans l'exemple ci-dessus).
 - Ces dépendances correspondent à d'autres règles dans le fichier Makefile, ou des fichiers sources.
 - Typiquement, une règle pour la construction d'un fichier *.o* dépend du fichier *.c* ou *.cpp* du même nom, ainsi que des headers *.h* duquel le code *.c* ou *.cpp* est dépendant, directement ou indirectement.
 - Les dépendances d'une règle se basent généralement sur la date de modification des fichiers : si la date d'une dépendance est postérieure à la target, alors la target doit être reconstruite.
 - Dans le cas d'une règle phony, les dépendances sont systématiquement exécutées.
- Une règle définit finalement un jeu de commandes (*command1*, *command2* dans l'exemple ci-dessus).
 - Typiquement des appels à [gcc](#) ou [g++](#), pour compiler les fichiers source en fichiers objets, ou pour linker finalement les fichiers objets en binaire exécutable.
 - Le jeu de commandes d'une règle n'est exécuté que si l'analyse des dépendances l'indique.
- Les règles peuvent être écrites de manière générique.
 - La caractère [%](#) peut être utilisé dans les targets et les dépendances.
 - Les variables spéciales [\\$@](#) et [\\$<](#) permettent respectivement de référencer la target et la dépendance dans le jeu de commandes d'une règle donnée.
- Les Makefiles permettent de définir des [variables](#).

- Ils mettent également à disposition un ensemble riche de **fonctions** permettant de calculer les variables de manière générique.

Ce système de dépendances par date permet d'optimiser les temps de compilation. En effet, seuls les fichiers objets pour les sources modifiés sont recompilés, puis l'application finale relinkée.

Problème : la gestion des dépendances inter-fichiers source étant elle-même dépendante des fichiers source eux-mêmes, toute modification des fichiers source nécessiterait de mettre à jour le fichier Makefile en conséquence... Fastidieux !

C'est pourquoi, on peut utiliser **gcc** pour générer une liste des dépendances de règles, qu'on n'a plus qu'à **inclure** dans notre fichier Makefile.

Problème résiduel : ce sous-Makefile de dépendances n'est pas généré automatiquement. Il faut donc rajouter une règle permettant de le mettre à jour à partir de l'état actuel des sources... Et il faut également penser à le committer dans l'historique git... Un peu mieux, mais ça reste encore fastidieux, et source d'erreurs.

TP : Syntaxe Makefile

Parcourir la ressource suivante pour une illustration rapide de la syntaxe Makefile : <https://dev.to/ashcript/comprendre-le-makefile-exemple-avec-le-langage-c-47n9>.

Astuce : Utiliser Makefile pour autre chose

...pour autre chose que des compilations de code.

Le système de gestion de dépendances par date peut être exploité pour d'autres usages impliquant des transformations de fichiers en d'autres :

- transformations XSL,
- productions documentaires,
- ...

2.4.4.2. CMake

Lorsqu'on peut en faire le choix, CMake constitue une alternative plus simple que les Makefiles pour les compilations C / C++.

Langage	Outil projet	Fichier projet	Documentation
C / C++	CMake	CMakeLists.txt	https://cmake.org/cmake/help/book/mastering-cmake/chapter/Writing%20CMakeLists%20Files.html

TP : CMake

Parcourir le tutoriel officiel proposé par CMake :
<https://cmake.org/cmake/help/latest/guide/tutorial/index.html#guide:CMake%20Tutorial>.

2.4.4.3. Des configurations de mieux en mieux intégrées

Avec les nouveaux langages, la mise en oeuvre du build de mieux en mieux intégrée au-travers des outils projet, et des configurations possibles dans les fichiers associés :

Langage	Outil projet	Fichier projet	Documentation
Java	Maven	pom.xml	https://maven.apache.org/guides/introduction/introduction-to-the-pom.html
Javascript	npm, yarn, pnpm	package.json	https://nodejs.org/api/packages.html
	Webpack	webpack.config.js	https://webpack.js.org/configuration/

Langage	Outil projet	Fichier projet	Documentation
	Metro	metro.config.js	https://metrobundle.dev/docs/configuration/
	esbuild	Configuration par API (4)	https://esbuild.github.io/api/
	Vite	vite.config.js	https://vite.dev/config/
Typescript	tsc	tsconfig.json	https://www.typescriptlang.org/tsconfig/
Rust	cargo	Cargo.toml	https://doc.rust-lang.org/cargo/reference/manifest.html , https://doc.rust-lang.org/cargo/reference/config.html
Go	go build	go.mod	https://go.dev/doc/modules/gomod-ref

i Notes

- (4) Pas vraiment de fichier de configuration pour esbuild. Pour configurer cet outil, la seule façon de faire est de créer un script launcher passant des configurations à esbuild de manière programmatique (par API (*Application Programming Interface*)).

2.5. Industrialisation

Afin d'industrialiser l'environnement de développement, on pourra reposer sur différents outils ou pratiques permettant d'assurer la maîtrise des éléments.

2.5.1. Pratiques d'équipe

Les revues de pairs contribuent grandement à la sécurisation des développements, et peuvent être outillées, notamment au-travers des pull-requests.

Les pull-requests sont des fonctionnalités attachées aux plateformes git, et font partie de la stratégie de branching du projet (cf. §5.1.2.3).

2.5.2. Règles de codage

Il est commun de définir des règles de codage pour assurer la qualité du logiciel, et notamment sa maintenabilité.

Pour ce faire, on pourra utiliser des outils tels que :

- des vérificateurs de typage : pour vérifier les règles de typage (mypy pour Python)
- des linters : pour vérifier des règles de codage (taille de fichier, taille de fonction, nombre de fonctions, cyclométrie, ...),
- des formatters : pour assurer une indentation homogène dans les fichiers source.

On pourra veiller à certaines règles de formattage pour assurer une meilleure interaction avec le suivi de version sous git, et limiter ainsi les merges conflictuels.

💡 Astuce : une ligne par élément

De sorte à limiter le contenu identifié comme diff sous git, on pourra suivre une première règle pour les paramètres de fonctions, ou items de tableaux et de dictionnaire, de passer à la ligne pour chaque élément.

Ainsi, une modification d'un nom de fonction, ou de nom de paramètre, ou de valeur de paramètre, ou une modification de la liste des éléments limitera l'identification du diff sur les seuls éléments modifiés, et non l'intégralité de la ligne.

Exemple :

Le diff


```
19c19
< _parser: argparse.ArgumentParser = argparse.ArgumentParser(description="Check cross
reference in a TOC enabled .md file.", add_help=False)
---
> _parser: argparse.ArgumentParser = argparse.ArgumentParser(description="Check cross
references in a TOC enabled .md file.", add_help=False)
```

pour le code Python suivant :

```
_parser: argparse.ArgumentParser = argparse.ArgumentParser(description="Check cross
references in a TOC enabled .md file.", add_help=False)
```

est moins clair que :

```
20c20
<     description="Check cross reference in a TOC enabled .md file.",
---
>     description="Check cross references in a TOC enabled .md file.",
```

lorsque le code est formaté comme suit :

```
_parser: argparse.ArgumentParser = argparse.ArgumentParser(
    description="Check cross references in a TOC enabled .md file.",
    add_help=False,
)
```

Le diff fait le focus sur la modification liée au paramètre `description` uniquement.

💡 Astuce : *final trailing comma*

Noter par ailleurs, dans l'exemple précédent, la virgule positionnée après le paramètre `add_help`.

Python autorise cette virgule optionnelle dans la syntaxe sur le dernier élément, virgule optionnelle qu'on positionne volontairement.

Cette *final trailing comma* permet d'assurer que la dernière ligne n'apparaîtra pas en diff si on rajoute des paramètres après le paramètre `add_help`.

Note : Tous les langages n'autorisent pas cette virgule finale, comme la syntaxe C par exemple, ce qui est bien dommage.

2.5.3. Git

L'utilisation de git constitue un socle quasiment systématique pour les développements logiciels aujourd'hui.

Git sert évidemment à suivre l'historique des fichiers source, mais également de tous les fichiers projet évoqués précédemment.

De fait, en tirant le dépôt du projet, on s'assure d'une répétabilité sur ces éléments.

Se reporter au §5.1.2 pour une découverte approfondie de l'outil git.

2.5.4. Automatisations

Dès lors qu'on peut automatiser des actions répétitives, cela constitue généralement des risques en moins d'erreurs manuelles.

2.5.4.1. Scripting

La mise en oeuvre de scripts constitue un bon moyen de sécuriser des pratiques.

Parmi les langages de scripting les plus utilisés pour le besoin, on peut citer :

- Bash
 - Langage de scripting associé à Linux, et notamment aux commandes Bash.
 - Disponible également sous Windows avec GitBash, invite de lignes de commandes installée avec git, ce qui est assez naturel pour du développement logiciel.
 - Syntaxe un peu complexe dès lors qu'on a besoin de faire des traitements algorithmiques.
- Python :
 - Plus facile que Bash lorsqu'on commence à avoir des traitements plus complexes.
 - Nécessite la disponibilité d'un interpréteur Python, et dans une version compatible avec les scripts développés.

👨‍💻 TP : Lignes de commande Bash

Parcourir les ressources suivantes pour une découverte des lignes de commandes Bash :

- <https://documentation.ubuntu.com/desktop/en/latest/tutorial/the-linux-command-line-for-beginners/>
- <https://linuxconfig.org/linux-commands-tutorial>

💡 Shebang

Sous Linux, la première ligne d'un script correspond habituellement à un *shebang*.

Shebang pour *She-Bang*, pour *#!*.

Permet d'indiquer à l'OS l'interpréteur avec lequel traiter le reste du script. Ainsi, si le script dispose des droits d'exécution, il se comporte comme un programme autonome. Il peut être appelé directement à partir de la ligne de commande Bash, ou à partir d'un autre script, sans avoir à savoir en quel langage il est implémenté.

Notes :

- L'extension du fichier (.sh, .py, ...) peut même être retirée.
- Sous GitBash, le simple fait de disposer d'un shebang rend un script automatiquement exécutable (émulation proposée par GitBash en l'absence de permissions de fichiers type Unix sous Windows).

On a coutume de passer par un appel `/usr/bin/env` pour une meilleure intégration avec l'OS.

Exemples :

- `#!/usr/bin/env bash`
- `#!/usr/bin/env perl`
- `#!/usr/bin/env python`

2.5.4.2. Chaîne CI - Continuous Integration

Une chaîne d'intégration continue (CI pour *Continuous Integration*) permet d'automatiser des scripts, ou pipelines, soit de manière chronique, soit sur occurrence d'événement.

Parmi les opérations classiquement automatisées en CI :

- vérifications de règles de codage,
- vérifications de typage,
- compilation,
- exécution des tests unitaires,
- exécution de tests de non-régression,
- exécution de campagnes de tests complètes,
- ...

Ces opérations peuvent être déclenchées :

- manuellement à la demande,
- de manière chronique : toutes les nuits, tous les weekends, ...
- avant ou après le merge d'une pull-request,
- ...

Le tableau ci-après liste quelques outils principaux de CI :

Source : <https://chat.mistral.ai/>, sous réserve de confirmation des informations.

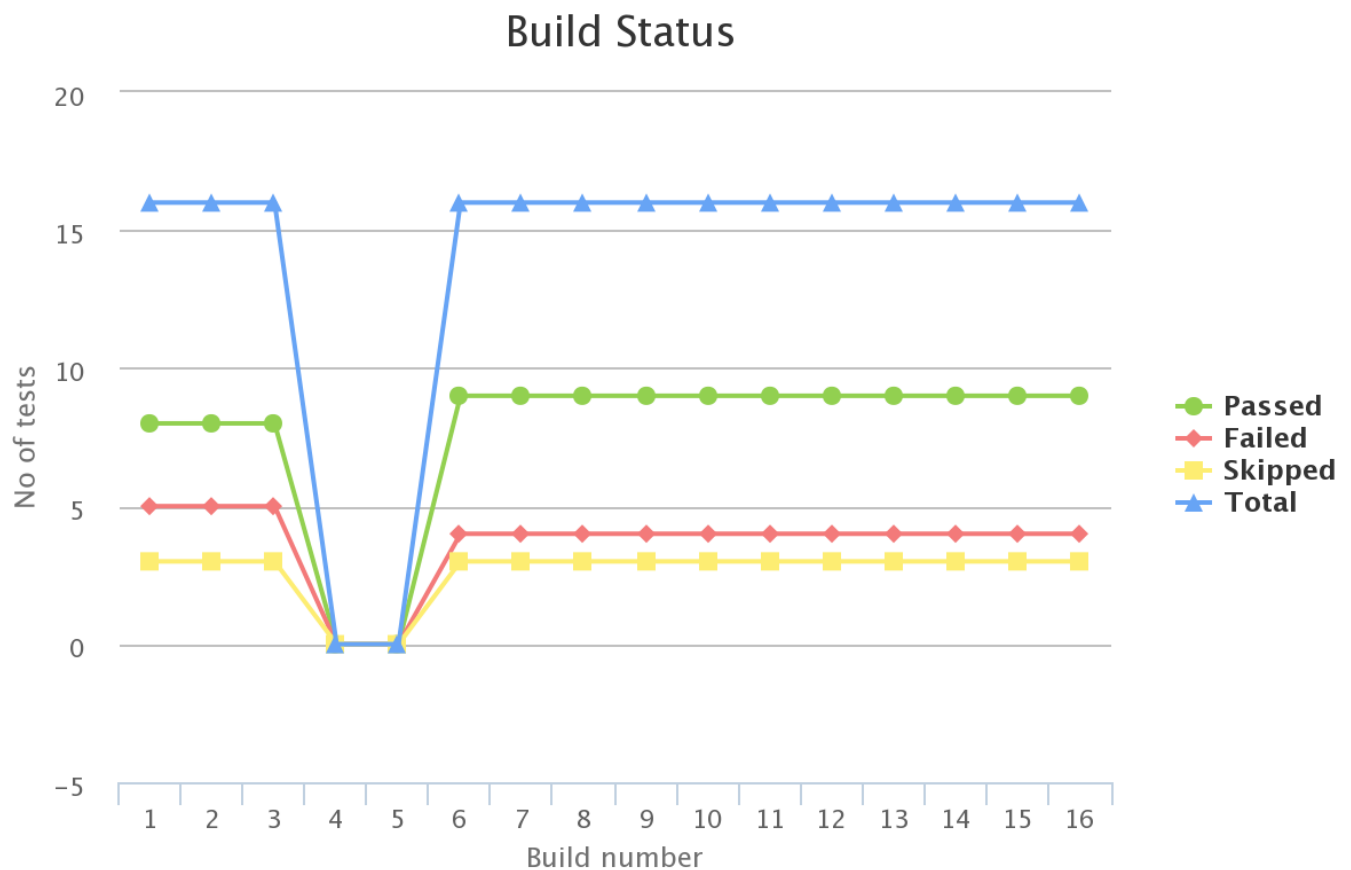
Solution	URL	Type de licence	Type de solution	Commentaires	Date
Jenkins	https://www.jenkins.io/	Open Source	On-premises (1)	Pionnier de la CI, nombreux plugins, coûts de maintenance et de configuration manuelle	2011
CircleCI	https://circleci.com/	Freemium	Cloud & on-premises	Simplicité et rapidité d'exécution, petits déploiements	2011
GitLab CI/CD	https://docs.gitlab.com/ci/	Open Source, Freemium	Cloud & on-premises	Intégré à GitLab, suite complète CI/CD, simplicité d'utilisation, moins performant pour des pipelines complexes	2015
Azure DevOps	https://azure.microsoft.com/en-us/products/devops/pipelines	Commercial	Cloud & on-premises	Solution Microsoft, suite complète CI/CD	2018
GitHub Actions	https://github.com/features/actions	Freemium	Cloud & on-premises (2)	Intégré nativement à GitHub, moins flexible	2019

i Notes

- (1) Un déploiement dans le cloud reste possible.
- (2) Voire hybride : possibilité de mixer cloud et on-premises.

Les outils de CI peuvent collecter des indicateurs qui permettent de suivre l'état de santé du projet dans le temps.

Chart	See children	Build Number → Package-Class-Testmethod names ↓	16	15	14	13	12	11	10	9	8	7	6	5
		org.common.samplea	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	N/A
		SampleATest	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	N/A
		testA	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	N/A
		testB	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	N/A
		testC	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	N/A
		testD	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	N/A
		org.common.sampleb	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	FAILED	N/A
		org.common.samplec	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	N/A
		SampleDTest	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	N/A
		testA	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	N/A
		testB	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	N/A
		testC	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	SKIPPED	N/A
		testD	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	PASSED	N/A

Source : <https://plugins.jenkins.io/test-results-analyzer/>Source : <https://plugins.jenkins.io/test-results-analyzer/>

👤 TP : Jenkins - CI (TODO)

👤 TP : [GitHub Actions](#) (réserver la partie CD pour le §4.3.1)

2.5.5. Conteneurs Docker d'outillages

La constitution d'images Docker embarquant des versions d'outils bien identifiées contribue à la maîtrise de l'environnement de développement.

Grâce à ces images, on peut répéter facilement une configuration de développement, pour chaque membre de l'équipe, ainsi que pour la chaîne de CI, et avec l'assurance d'utiliser les mêmes versions des outils.

Cette approche a par ailleurs l'avantage de pouvoir être suivie dans l'historique git du projet par l'intermédiaire de Dockerfiles.

👤 TP : [Docker tool Maven JDK](#)

2.5.6. Documentation

A défaut, pour tout ce qu'on n'aura pas réussi à sécuriser avec des moyens techniques, ou quand bien même ! on n'oubliera pas d'être prolixe en documentation utile.

En matière d'industrialisation, on pourra rédiger :

- des starter kits de documentation, pour la montée en compétence rapide des nouveaux arrivants,
- des guides d'installation, pour la mise en place de l'environnement de développement,
- des procédures sous forme de checklists, pour cadrer les opérations sensibles (livraison, ...),
- ...

Se reporter au §5.3 pour des conseils sur la gestion de la documentation.

3. Exécution

Dès les premières versions de notre logiciel disponibles, il s'avère intéressant de savoir exécuter notre production aux différentes étapes de sa construction :

- en cours de développement, pour des tests unitaires, ou tests d'intégration libres,
- sur des versions intermédiaires, pour des tests de non-régression quotidiens par exemple,
- sur des versions identifiées, pour des campagnes de test avant livraison.

3.1. Exécution des applications PC

Le cas le plus simple : on développe en général sur une machine étant directement la cible du logiciel développé.

Restent les problématiques d'OS : Windows, Linux, Mac.

Apple reste la plateforme la plus fermée en la matière : pour développer des applications pour MacOS, il faut compiler avec Xcode sur un poste Mac, pour pouvoir ensuite pousser une appli dans l'App Store.

Pour pallier cette problématique, on pourra :

- utiliser des machines virtuelles, en local ou dans le cloud,
- disposer de différentes machines sur les différents OS.

3.2. Exécution des applications Web

Pour le cas des applications Web, la difficulté supplémentaire réside dans le fait qu'on doit faire tourner un ensemble de services constituant le système d'information : serveur HTTP et base de données a minima.

3.2.1. Exécution locale

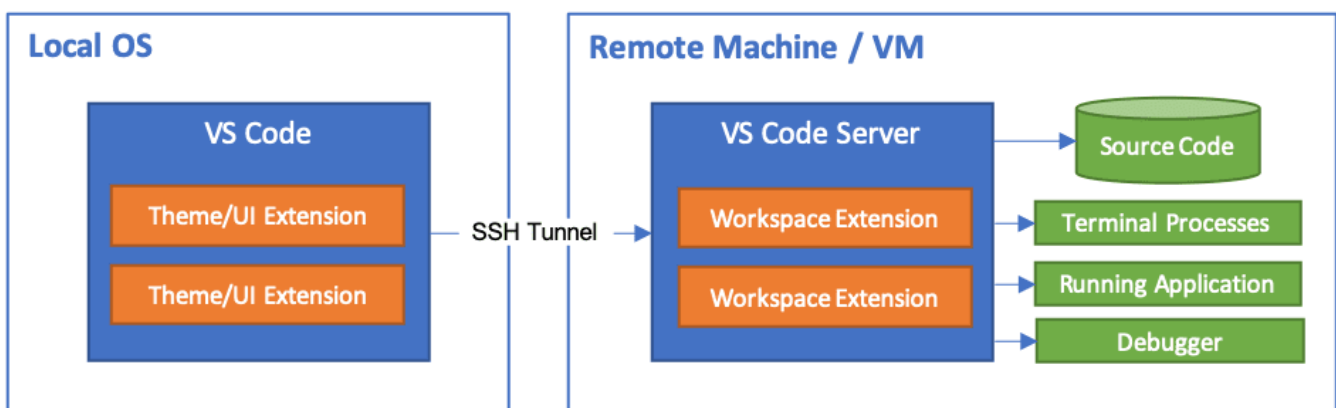
Première option : lancer en local tous les services, et les faire interopérer.

Les packs LAMP et WAMP (Linux/Windows Apache MySQL PHP) ont été populaires pour leur simplicité d'installation et d'administration.

3.2.2. Exécution distante + développement en Remote SSH

Si on a accès en SSH à une machine distante exécutant notre code, on peut aussi travailler en *remote SSH* :

- Le code source est hébergé et s'exécute sur la machine distante.
- On dispose d'un IDE sur le poste local.
- L'IDE travaille sur les fichiers source à distance via le lien SSH.



Source : <https://code.visualstudio.com/docs/remote/ssh>

💡 Astuce : Extension VS Code Remote - SSH

L'extension VS Code [Remote - SSH](#) permet ce travail à distance sur un code source.

Cf. <https://code.visualstudio.com/docs/remote/ssh>.

La machine distante pouvant être :

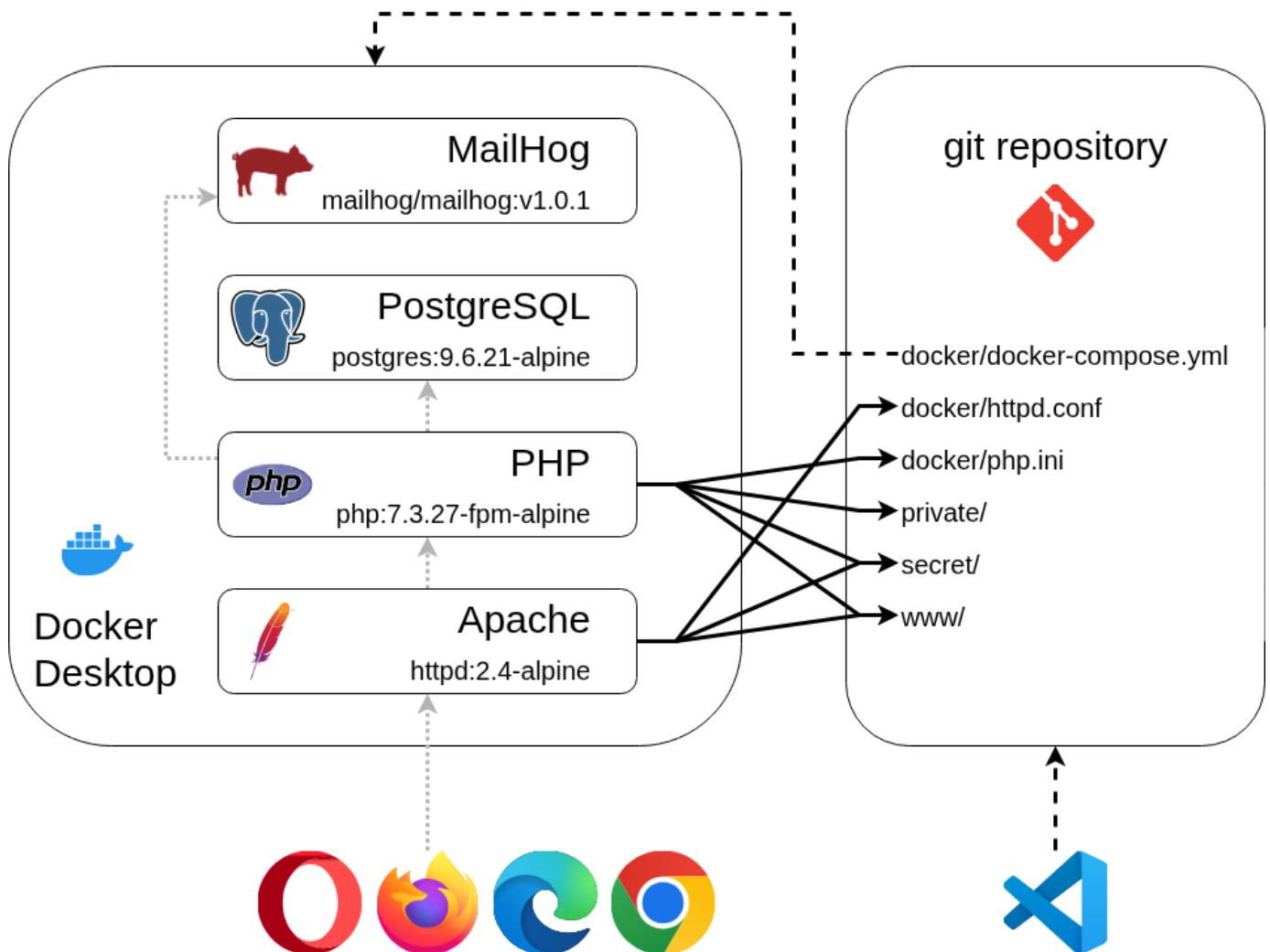
- une machine physique, mais c'est de plus en plus rare aujourd'hui ;
- une machine virtuelle
 - tournant on-premises sur une solution de virtualisation telle VMware ou Proxmox,
 - ou dans le cloud ;
- des conteneurs Docker, c'est moins probable, car on basculera certainement alors sur la stratégie d'exécution suivante.

👤 TP : [Remote SSH VS Code Maven JDK](#)

3.2.3. Conteneurs Dockers locaux

Depuis le développement de la conteneurisation, le déploiement de différents services avec Docker offre une alternative intéressante :

- Des conteneurs Docker sont lancés pour chacun des services : serveur HTTP, base de données, ... voire moteur CGI PHP dans un conteneur dédié.
- Le code source reste sur la machine hôte, et monté dans les conteneurs au moyen de *volumes*.



Ce type de déploiement apporte un certain nombre d'avantages :

- Les configurations Docker sont gérées en local, et suivies sous git.
- On peut travailler à partir d'un dépôt git unique (monorepo) simplifiant la gestion de configuration (par rapport à la gestion de plusieurs dépôts pour chacun des services).

- On peut s'arranger pour suivre exactement la configuration des versions cibles : serveur HTTP, Java, PHP, base de données.
- Le lancement des différents services est scriptable, permettant un usage facilité au quotidien.

🧑🏻‍💻 TP : Développement Web avec Docker (TODO)

Mémo :

- Reproduction de la config cible de l'hébergeur.
- <https://www.docker.com/blog/docker-for-web-developers/>
- <https://www.geeksforgeeks.org/blogs/how-to-use-docker-for-web-development/>

3.2.4. Chaîne CD sur plateforme de test

Solution la plus riche, car elle nécessite :

- la mobilisation de ressources matérielles pouvant héberger l'exécution de machines virtuelles, typiquement des PC bare-metal, en baie, avec VMware ou Proxmox ;
- le déploiement d'une solution de CD (cf. §4.3.1) pour instancier des POD Docker pour nos services ;
- et probablement une solution de logging permettant d'investiguer plus efficacement en cas de problème (cf. §3.5).

Moins réactif que l'option précédente, car nécessite des redéploiements de conteneurs Dockers.

Adapté pour des phases de test / validation.

3.3. Exécution des applications mobile

D'expérience personnelle, l'exécution des applications mobiles constitue une difficulté en développement.

3.3.1. Exécution sur smartphone physique

La première option pour exécuter une application mobile est de la tester sur un terminal matériel.

Ca fonctionne, mais c'est extrêmement cher dans l'absolu, car il faudrait acheter un certain nombre de modèles de téléphones ou tablettes.

3.3.2. Emulateurs sur poste de développement

Il existe des émulateurs qui permettent de simuler le fonctionnement d'un terminal mobile :

- Android :
 - Emulateurs basés sur la technologie Qemu.
 - A configurer et lancer à partir de Android Studio (1).
- iOS :
 - Testflight, <https://testflight.apple.com/> (pas eu l'occasion de tester, à confirmer).

Parmi les intérêts de ces émulateurs :

- coût 0, pas besoin d'acheter de terminaux hardware,
- capacité à émuler un grand nombre de modèles (type de processeur, taille d'écran, ...).

Ces émulateurs peuvent toutefois présenter des limitations de fonctionnement, et surtout patissent généralement de lenteurs qui pénalisent l'activité de développement.

⚠️ (1) Retour d'expérience Android Studio mitigé

Cela n'engage que moi, mais je n'ai pas un très bon retour d'expérience avec l'IDE Android Studio :

- C'est de mémoire assez lent, ce qui rajoute à la lenteur des émulateurs.
- Les configurations sont assez complexes à réaliser.

Travaillant en React Native, je travaillais principalement avec VS Code, et ne lançais Android Studio qu'en cas de nécessité.

3.3.3. Emulateurs dans le cloud

En cherchant rapidement sur Internet, on trouve des noms de plateformes permettant d'émuler des terminaux mobiles (LDCloud, RedFinger, ...).

On trouve également des plateformes de CI/CD dédiées aux développements mobiles (Bitrise, ..., cf. §4.3.1).

L'avantage que l'on peut attendre de ces plateformes par rapport à un PC local est que les hébergeurs peuvent avoir dimensionné les configurations matérielles permettant de supporter les contraintes liées à ces technologies d'émulation.

Je n'ai personnellement pas eu l'occasion de tester ces solutions. Solutions qui fonctionnent certainement, mais certainement pas gratuites non plus.

3.3.4. Expo & Expo Go

A noter finalement l'existence de la solution Expo (<https://expo.dev/>).

Expo est basé sur React Native (<https://reactnative.dev/>), un framework cross-platform permettant de développer des interfaces pour différentes cibles :

- Android,
- iOS,
- Applications Web.

Expo propose un mode Expo Go (<https://expo.dev/go>) qui consiste à ne construire que le Javascript bundlé, interprété ensuite par un moteur Expo Go, soit sur téléphone, soit en ligne (à confirmer).

Cette option peut permettre d'aller plus vite sur le développement initial de l'application.

Une fois l'application développée, il ne reste plus qu'à construire l'application finale, exécutable sans nécessiter le moteur Expo Go (à confirmer). Cf. <https://docs.expo.dev/develop/development-builds/expo-go-to-dev-build/>.

3.4. Exécution des logiciels embarqués

3.4.1. Exécution sur cible hardware

Pour l'exécution des logiciels embarqués sur la cible hardware, il faut commencer par charger le logiciel à exécuter sur la cible.

3.4.1.1. OS riche

Lorsqu'on est sur des cibles avec un OS riche, tel Raspbian/Linux sur Raspberry, le chargement du logiciel sur la cible peut se faire relativement facilement, via SSH / scp notamment.

Dans ce type de déploiement, lancer le logiciel est assez direct également.

i OS embarqué

Dans les autres cas présentés après, c'est souvent notre logiciel qui embarque l'OS : Linux (avec buildroot), vxWorks, µC-OS, ...

3.4.1.2. Bootloader

Il existe également des cas où notre logiciel embarqué peut être lancé par un bootloader.

C'est typiquement le cas des images Linux construites avec buildroot, et qu'on pourra lancer avec un bootloader UEFI présent sur la carte.

Un bootloader UEFI permet de lancer une image depuis une partition disque, mais également depuis un device USB, ou en TFTP depuis un serveur, ce qui offre des facilités d'exécution sur la cible.

💡 Dev "à chaud" avec Linux embarqué

Et une fois qu'on a notre image Linux qui tourne, on peut aussi faire des modifications "à chaud" :

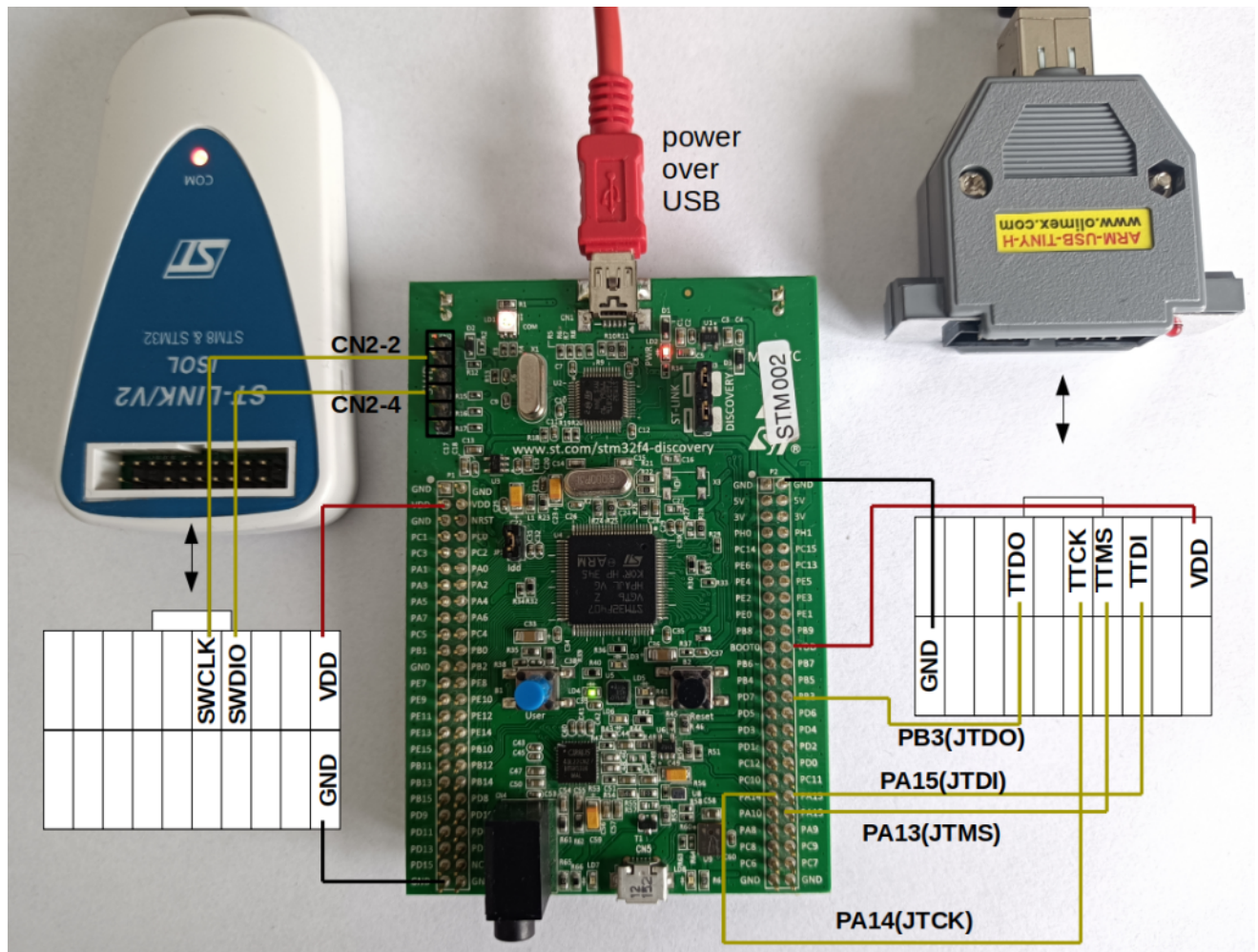
- modifier des scripts et des configurations avec `vi`,
- exécuter des commandes système,
- recompiler et upgrader via SSH certains programmes embarqués.

3.4.1.3. Programmation mémoire

Il existe enfin des cas où notre logiciel embarqué est directement chargé par le processeur, depuis un composant mémoire afférent sur la carte. Il faut donc programmer ce composant mémoire sur la carte.

Pour ce faire, deux options à ma connaissance :

- **Utilisation d'une sonde branchée sur un connecteur JTAG**



Source : <https://www.actuatedrobots.com/debugging-with-jtag/>

Une sonde JTAG est un matériel spécifique (tel les sondes Lauterbach - <https://www.lauterbach.com/>) qui permet de piloter le processeur sur une carte.

Elle permet de charger le logiciel à exécuter, exécuter des commandes processeur, et déboguer l'exécution du logiciel sur le processeur.

On peut ainsi charger le logiciel et le déboguer directement, ou le charger et l'écrire dans une mémoire afférente au processeur.

- **Programmation de la mémoire cible à l'aide d'une fonction logicielle**

Les sondes JTAG étant du matériel spécifique, potentiellement cher, on pourra chercher à se passer de celles-ci au quotidien.

Pour ce faire, on pourra développer des fonctions logicielles, exécutées par le processeur lui-même ou d'autres processeurs sur la carte, permettant de reprogrammer la mémoire contenant le logiciel à exécuter.

Pour le transfert du logiciel, selon les interfaces disponibles sur la carte, on pourra utiliser :

- des ports USB permettant de brancher une clé,
- des liens réseaux : FTP, TFTP, SSH, ...
- un lien série : avec utilisation d'un protocole Xmodem, Ymodem, Zmodem, Kermit... pour le transfert des données binaires.

On programme, on reboote, et on croise les doigts pour que ça reboote correctement. Si ça ne reboote pas, il faudra revenir à la sonde.

Dans tous les cas, l'exécution sur cible hardware requiert la disponibilité de la cible hardware elle-même, voire d'une sonde JTAG spécifique pour le processeur cible. Du matériel qui selon les cas peut se chiffrer à plusieurs milliers d'euros.

3.4.2. Exécution dans un émulateur

De sorte à réduire les coûts, on peut aussi utiliser des émulateurs de processeurs.

Qemu (<https://www.qemu.org/>) permet de mettre en oeuvre de tels émulateurs.

Cela nécessite toutefois un peu de travail préliminaire pour bien configurer son émulateur de sorte à ce qu'il corresponde à la cible hardware visée.

Cela nécessite également du travail d'outillage autour du Qemu pour émuler toutes les interfaces d'entrée / sortie du processeur.

⚠ Lenteur des émulateurs

On note un problème récurrent de lenteur d'exécution des émulateurs.

Ce problème peut s'avérer assez pénalisant pour la productivité des activités de développement et intégration, ainsi que des activités de tests également.

💡 Snapshots Qemu

Il existe une fonction de *snapshot* avec Qemu.

Je n'ai pas eu l'occasion de tester, mais cela peut probablement accélérer le démarrage des tests dans des conditions initiales données. On pourrait effectivement envisager constituer des snapshots pour chaque conditions initiales identifiées dans le plan de test.

3.4.3. Exécution en mode simulé X86

L'idée ici n'est pas d'exécuter le logiciel sur la cible, ou un émulateur équivalent à la cible, mais de le faire tourner directement sur la machine de développement en X86.

Ce n'est donc probablement pas représentatif de l'architecture hardware cible (32/64 bits, big/little endian), mais cela peut permettre de pré-debugger déjà une bonne partie du fonctionnel avant de finir par une intégration hard/soft finale sur la cible.

Cette solution peut également présenter l'avantage de simplifier l'effort d'outillage pour les interfaces d'entrée / sortie, car on peut décider de détourner le périmètre de simulation là où c'est le plus efficace.

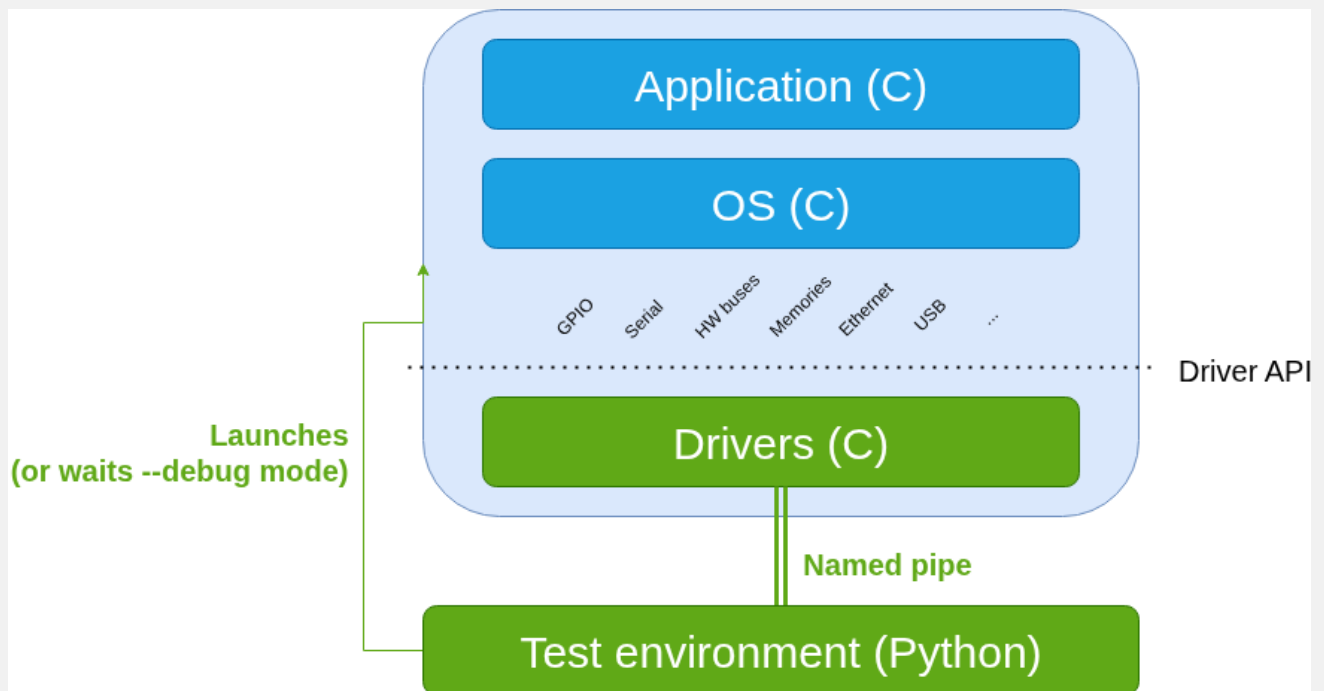
💡 Retex simulation X86 / instrumentation des API drivers

J'ai eu l'occasion de mener une stratégie de test avec un logiciel simulé tel que décrit ici, expérience couronnée d'un certain succès soit dit en passant.

Le logiciel simulé X86 embarquait :

- Un OS minimaliste,
- Le code applicatif utilisant l'API des drivers.

L'implémentation des drivers était remplacée par du code d'outillage permettant une interaction avec un environnement de test en Python.



Comme indiqué, l'expérience avait été couronnée d'un certain succès, notamment pour les raisons suivantes :

- **Facilité de mise en oeuvre** : Le développement du code d'outillage remplaçant les drivers était d'une complexité abordable. Pas besoin de descendre dans des spécifications matérielles pour émuler les entrées / sorties, on reste sur un niveau programmatique d'API drivers.
- **Facilité de déploiement** : Le code X86, une fois compilé, est directement utilisable par l'environnement de test. Bien plus rapide que de charger le logiciel sur la carte pour chaque test qu'on veut réaliser.
- **Facilité de reproduction des problèmes** : Grâce au couplage avec l'environnement de test, il est facile de rejouer un test en erreur, ou d'adapter un test existant, pour reproduire un problème sur lequel on investigate.
- **Facilité de debugging** : Une fois le problème reproduit avec l'environnement de test, il est plus rapide de déboguer le code directement à partir de la machine de développement que sur une cible hardware distante.
- **Rapidité d'exécution des tests** : En évitant des temps de chargement, et des temps de reset hardware, une campagne de tests se trouve grandement accélérée. Le déroulement d'une campagne de non-régression de quelques centaines de test passait en 10-20 minutes seulement.

3.5. Stratégie de logs

On a vu précédemment qu'on pouvait déboguer les logiciels en exécution, mais que le debugging avait ses limites (cf. §2.2).

Il reste effectivement judicieux de mettre en place une stratégie de logs dès le début du projet, avec quelques caractéristiques utiles :

- Format des logs : simple texte, syslog, JSON, ...
- Données : informations enregistrées dans les logs.
- Datation : T0, synchronisation (NTP), précision.
- Niveaux de criticité : debug, info, warning, erreur classiquement.
- Colorisation : mettre en évidence les erreurs et warning notamment.
- Indentation : utile pour analyser les logs de traitements récursifs.
- Filtrages : par fonction, par module, configuration.

L'utilisation d'outils tels que Kibana peut s'avérer utile pour exploiter ces logs.

Le tableau ci-après donne une liste d'outils de gestion des logs parmi les plus connus :

Source : <https://chat.mistral.ai/>, sous réserve de confirmation des informations.

Solution	Type de licence	Type de solution	Commentaires	Date
Splunk	Commercial	On-premises & cloud	Orientée sécurité, langage de requête spécifique	2003
Datadog	Commercial	Cloud	Référence du marché	2010
Graylog	SSPL, commercial	On-premises & cloud	Robuste et conviviale, moins adaptée aux grands volumes	2011
ELK (Elasticsearch, Logstash, Kibana)	AGPLv3, SSPL, commercial	On-premises & cloud	Open Source, flexible, adaptée aux grands volumes, coûts de mise en œuvre technique	2012
Grafana Loki	Open Source, Freemium	On-premises (1)	Extension Grafana (métriques) pour gestion de logs, moins mature	2018

i Notes

- (1) S'installe en tant qu'extension de Grafana, donc plutôt on-premises.

4. Delivery / déploiement

Après avoir développé le logiciel, puis s'être doté des moyens de l'exécuter (notamment pour le tester), vient le moment de le livrer, voire de le déployer.

On identifie une première étape d'enregistrement du livrable, probablement sur des moyens internes. Puis on détaille la livraison du logiciel produit, dépendante du type de logiciel avant tout.

4.1. Enregistrements de livrables

Une des premières actions de livraison, consiste à enregistrer proprement le logiciel produit.

Le logiciel produit est rarement le code source tel que, mais plutôt :

- le produit d'une compilation,
- ou la livraison sous la forme d'un package,
- ou la livraison sous la forme d'une image Docker,
- ...

Cet enregistrement se fait généralement sur des moyens internes.

Pour ce faire, différentes options possibles.

4.1.1. Répertoires partagés, GED

Une des premières options consiste à utiliser les espaces partagés pour l'enregistrement des documents du projets.

Ces espaces peuvent être des répertoires partagés, ou un outil de GED (Gestion Electronique des Documents) tels Sharepoint, ...

C'est une solution assez naturelle, car c'est le prolongement des autres enregistrements (de documents notamment) déjà réalisés pour le compte du projet.

Cette solution a toutefois l'inconvénient d'enregistrer un volume de données conséquent dans un espace pas toujours adapté. En effet, les logiciels produits peuvent se présenter sous la forme de fichiers binaires volumineux. Dans tous les cas, à force des versions successives, cela finit souvent par représenter des volumes de données conséquents.

Les espaces partagés finissent régulièrement saturés dans le temps, et du ménage doit souvent être réalisé sur ces espaces.

4.1.2. Enregistrement sous git

Lorsque cela s'y prête, on peut envisager d'enregistrer le logiciel produit sous git, comme une librairie JS bundlée par exemple.

Mais c'est rarement une option adaptée sinon, git étant avant tout fait pour enregistrer des fichiers sources, et non des fichiers binaires.

4.1.3. Dépôt de binaires

Il existe également des outils permettant d'enregistrer des fichiers binaires potentiellement volumineux, tels que :

- des programmes d'installation,
- des images de conteneurs ou de machines virtuelles,
- des résultats de compilations,
- des données de test,
- des configurations,
- des archives,
- ...

On trouve plusieurs noms possibles pour ce type d'outil :

- Binary Repository Manager,
- Artifact Repository,
- Package Repository,
- Software Repository,
- ...

TODO : Tableau des outils du marché

Lorsqu'on dispose d'un tel outil, alors c'est un espace de stockage particulièrement adapté pour les enregistrement des versions logicielles produites.

4.1.4. Enregistrement dans les dépôts publics

Lorsque le projet s'y prête, on peut également enregistrer les versions logicielles dans les dépôts publics :

- JS, sur npmjs.org
- Python, sur pypi.org
- PHP, sur packagist.org
- Rust, sur crates.io
- Docker, sur Docker Hub
- ...

Un enregistrement dans les dépôts publics constitue une forme de livraison en soit, pour un public de développeurs.

👤 ✂ TP : Publication de logiciel JS sur npmjs.org (TODO)

👤 ✂ TP : Publication de logiciel Python sur pypi.org (TODO)

Mémo :

- `python -m build`
- <https://realpython.com/pypi-publish-python-package/#publish-your-package-to-pypi>

👤 ✂ TP : Publication de logiciel PHP sur packagist.org (TODO)

Mémo :

- <https://packagist.org/> => §"Publishing Packages"
- <https://packagist.org/about> => §"How to submit packages?"

👤 ✂ TP : Publication de logiciel Rust sur crates.io (TODO)

👤 ✂ TP : Publication d'une image Docker sur Docker Hub (TODO)

4.2. Applications PC

La livraison des applications PC dépend avant tout du système d'exploitation cible.

4.2.1. Applications Windows

La livraison d'une application Windows passe généralement par la génération d'un programme d'installation (ou *setup*).

Pour produire des programmes d'installation, plusieurs outils possibles :

- InstallShield (<https://www.revenera.com/install/products/installshield/>),
- Inno Setup (<https://jrsoftware.org/isinfo.php>),
- InstallForge (<https://installforge.net/gallery/>),
- Wix (<https://www.firegiant.com/wixtoolset/>),
- ...

Il semble qu'aujourd'hui l'IDE Visual Studio permette de générer directement des programmes d'installation (à confirmer).

🧑🏻‍💻 TP : Génération de programme d'installation avec Inno Setup (TODO)

🧑🏻‍💻 TP : Génération de programme d'installation avec Visual Studio (TODO)

Pour autant, une fois le programme d'installation généré, on n'identifie pas de canal de distribution de référence pour distribuer un logiciel Windows (1).

Il existe effectivement le *Microsoft Store*, pour le cas spécifique des *Windows Apps*, mais il ne semble pas que cela constitue un canal de distribution majeur (1).

Les logiciels restent aujourd'hui majoritairement téléchargeables via des sites non gérés par Microsoft (1) :

- SourceForge (<https://sourceforge.net/>),
- Clubic (<https://www.clubic.com/telecharger>),
- Steam (<https://store.steampowered.com/>, spécialisé dans la diffusion de jeux vidéos),
- ...

⚠ (1) Disclaimer

Sur la base d'observations personnelles. A étayer.

Il existe également quelques initiatives de distributions de packages "à la Linux" pour Windows :

- Chocolatey (<https://chocolatey.org/>),
- WinGet (<https://learn.microsoft.com/fr-fr/windows/package-manager/winget/>),
- ...

L'utilisation de ces systèmes reste toutefois confidentielle, limitée essentiellement à un usage de développement.

4.2.2. Applications Linux

Les logiciels Linux sont essentiellement distribués via des dépôts de packages.

Deux grands systèmes de packages principaux sous Linux :

- **Packages .deb** : Packages Debian. Téléchargés et installés à l'aide de la commande **apt** (*Advanced Package Tool*, <https://manpages.debian.org/trixie/apt/apt.8.en.html>).
- **Packages RPM** : Initialement *RedHat Package Manager* (<https://rpm.org/>).

🧑🏻‍💻 TP : Génération d'un package .deb (TODO)

🧑🏻‍💻 TP : Génération d'un package RPM (TODO)

Depuis quelques temps, Canonical (société mettant à disposition Ubuntu) a également développé le système *snap* (<https://snapcraft.io/>), un système de packages conteneurisés.

i Intérêt d'un système de packages conteneurisés

L'intérêt de conteneuriser les installations est certainement de limiter les conflits potentiels entre les différentes applications installées sur un même poste (à confirmer).

4.2.3. Applications Mac

! Distribution d'applications Mac (TODO)

N'ayant pas d'expérience significative dans le développement d'applications Mac, cette section reste à documenter.

4.3. Applications Web

En règle général, lorsqu'on développe une application Web, on l'opère soi-même.

La livrer signifie donc la déployer en production.

4.3.1. Chaîne CD - Continuous Delivery ou Continuous Deployment

Dans la continuité de la CI (*Continuous Integration*, cf. §2.5.4.2) on parle de CD, pour *Continuous Delivery* ou *Continuous Deployment*.

L'objectif est de déployer le plus simplement possible les services Web développés, à l'aide de configurations et quelques clics.

Un bon outil de CD permet également de monitorer efficacement le parc des services déployés.

Le tableau suivant présente un aperçu des principaux outils du marché.

Sources : <https://thectoclub.com/tools/best-ci-cd-tools/> + <https://chat.mistral.ai/>, sous réserve de confirmation des informations.

Solution	URL	Type de licence	Commentaires	Date
Jenkins	https://www.jenkins.io/	Open Source (MIT)	L'un des outils CI/CD les plus anciens et les plus flexibles. Extensible via des plugins. Nécessite maintenance et configuration manuelle importantes.	2011
CircleCI	https://circleci.com/	Freemium	Rapidité et facilité d'utilisation, notamment dans les environnements cloud. Capacités de parallélisation et d'intégration avec Docker et Kubernetes.	2011

Solution	URL	Type de licence	Commentaires	Date
Octopus Deploy	https://octopus.com/	Commercial	Déploiement automatisé et gestion des releases. Particulièrement adapté .NET et Windows. Déploiements complexes et multi-environnements, sécurité et conformité.	2012
GitLab CI/CD	https://docs.gitlab.com/ci/	Open Source, Freemium	Intégré nativement à GitLab. Approche déclarative et intégration native Kubernetes.	2014
Terraform	https://developer.hashicorp.com/terraform	Open Source (MPL), commercial	Outil d'infrastructure as code (IaC, pas CI/CD à proprement parler). Permet de provisionner et de gérer des infrastructures multi-cloud de manière déclarative.	2014
Bitrise	https://bitrise.io/	Commercial	Spécialement conçu pour le développement mobile.	2015
Google Cloud Build	https://cloud.google.com/build	Commercial	Solution CI/CD Google Cloud.	2018
Azure DevOps	https://azure.microsoft.com/en-us/products/devops	Commercial	Solution CI/CD Microsoft Azure.	2018

Solution	URL	Type de licence	Commentaires	Date
OpenShift Pipelines	https://docs.redhat.com/en/documentation/red_hat_openshift_pipelines	Open Source (Apache 2.0), commerciale	Solution CI/CD native pour Kubernetes, basée sur Tekton. Solution sécurisée, scalable et intégrée à une infrastructure conteneurisée.	2019
Argo CD	https://argoproj.github.io/cd/	Open Source (Apache v2)	GitOps continuous delivery tool for Kubernetes	2019
GitHub Actions	https://github.com/features/actions	Commercial	Directement intégré dans GitHub. Permet d'automatiser les workflows CI/CD directement depuis les dépôts GitHub.	2019

i DevOps

Le DevOps est avant tout une culture de rapprochement des équipes de développement d'une part (dev) et des équipes opérationnelles d'autre part (ops).

La culture DevOps prône également l'automatisation et le monitoring de toutes les étapes de la création d'un logiciel jusqu'à son déploiement.

En ce sens, les outils de CD contribuent à la démarche DevOps.

🧑🏻‍💻 TP : [GitHub Actions](#) (partie CD)

🧑🏻‍💻 TP : Déploiement de services Web avec OpenShift (TODO)

4.3.2. Infrastructure As Code (IaC) / GitOps

Dans sa version ultime, la CD reposera uniquement sur des configurations enregistrées sous git, les opérations manuelles en prod étant proscrites.

On parle alors d'approche *Infrastructure As Code (IaC)* ou GitOps.

Cette stratégie permet d'assurer un grand niveau de maîtrise des éléments déployés :

- le code de l'application Web, géré sous git,
- les configurations associées, gérées sous git également,
- les configurations de conteneurisation (DockerFiles, docker-compose, ou autres configurations en fonction de l'outil de CD), gérées sous git également,
- les configurations réseaux, comme celles des WAF (*Web Application Firewall*), gérées sous git également,
- ...

En effet, avec cette stratégie, on a la connaissance de ce qui tourne en prod à partir du dépôt, les configurations qui ont évolué et pour quels motifs.

4.3.3. Points de vigilance

A propos des mises en production, quelques points de vigilance sont à observer.

⚠ Compatibilité des données

En cas d'évolution du schéma des données, la mise en production ne signifie pas seulement mettre en service le nouveau logiciel, mais également assurer la compatibilité des données N - 1 pour la nouvelle version N déployée.

Pour ce faire, cela passe généralement par des scripts de migration des données.

⚠ Continuité de service

Dans tous les cas, la question de la continuité de service peut se poser :

- Peut-on arrêter la version N - 1, pour redémarrer le service ensuite avec la nouvelle version N ?
- Ou doit-on s'assurer de la continuité du service pendant la mise en production ?

L'article <https://blog.cellenza.com/cloud-2/strategie-de-ci-cd-sur-kubernetes/> identifie différentes stratégies de déploiement :

- Rolling Update
- Blue-Green
- Canary Release
- A/B Testing

4.3.4. Scalabilité

Pour terminer ce chapitre sur le déploiement des applications Web, on évoque la question de la scalabilité des systèmes d'information, c'est-à-dire la possibilité d'augmenter la capacité de traitement du système d'information.

Quelques points de design permettent de tendre vers cette scalabilité :

- Respecter un **découpage en couches**, permettant de faciliter la maintenance et l'évolutivité du système.
- Privilégier les **traitements stateless**, tels que les API REST. On peut ainsi multi-instancier les PODs de traitement derrière une adresse virtuelle.

Cette scalabilité représente un atout pour le système d'information, capable d'évoluer vers plus de performances. Mais c'est dans le même temps une contrainte pour les activités de déploiement, car on a dès lors autant de PODs que de couches considérées, et potentiellement multipliés par le nombre de PODs en parallèle pour les services multi-instanciés.

Les exigences de scalabilité justifient d'autant l'adoption d'un outil de CD pour pouvoir gérer plus simplement tous les PODs déployés.

4.3.5. Pour en savoir plus

Pour en savoir plus sur les stratégies de CD, se reporter au très bon article déjà cité précédemment : <https://blog.cellenza.com/cloud-2/strategie-de-ci-cd-sur-kubernetes/>.

4.4. Applications mobiles

Les déploiements des applications mobiles passent par la publication dans les stores officiels :

- Play Store pour les applications Android,
- App Store pour les applications iOS.

⚠ Gestion des comptes de publication

Des comptes étant requis pour publier les applications dans les stores officiels, cela peut amener à une certaine difficulté sur la gestion de ces comptes :

- quelle adresse email utiliser ?
- qui détient le mot de passe ?
- ...

De sorte à ce que le compte puisse être attaché au produit et à l'entreprise, et non à un individu pouvant sortir du projet à un moment donné, il faudra certainement créer une adresse email dédiée.

Pour la confidentialité du mot de passe, on pourra chercher à s'appuyer sur des outils de gestion de secrets, tels Vault, ...

4.5. Logiciels embarqués

Pour les logiciels embarqués, il n'existe pas vraiment de canal de référence pour le déploiement du logiciel.

4.5.1. Programmation initiale

Au début du cycle de vie du produit, le logiciel est programmé en usine, par une opération diverse, en fonction du hardware cible :

- écriture d'un composant mémoire par un matériel spécifique avant soudure sur la carte,
- réécriture d'un logiciel de boot par procédure de flashage,
- chargement d'un binaire par TFTP,
- ...

4.5.2. Mises à jour logicielles

Ensuite, pour les mises à jour logicielles, cela dépend de la solution développée.

Pour les systèmes non connectés ou non administrés, la mise à jour à distance n'est pas possible. Dès lors, tout besoin de mise à jour logicielle nécessite un *retour usine*, contraignant et généralement coûteux.

Exemples de systèmes non connectés ou non administrés, nécessitant des retours usine :

- le logiciel gérant les programmes d'un lave-linge,
- le logiciel d'ABS sur un modèle de voiture,
- ...

Les systèmes connectés et administrés prévoient généralement une fonction de mise à jour logicielle à distance : le logiciel est envoyé par réseau à l'équipement, qui le programme dans sa mémoire, puis redémarre avec le nouveau logiciel.

Exemples de systèmes administrés à distance :

- boxes Internet,
- chiffreurs IPsec Mistral (<https://www.youtube.com/watch?v=OyP1JigSLgg>),
- ...

⚠ Précautions sur mises à jour logicielles à distance

Pour éviter le plantage à distance, et l'obligation d'un retour usine coûteux, surchargé de l'insatisfaction client dû à l'indisponibilité du service, les fonctions de mise à jour logicielle à distance doivent anticiper quelques cas d'erreurs possibles :

- Vérification de l'intégrité du logiciel reçu (voire de la signature, pour éviter des attaques), avant de tenter de le programmer et de redémarrer dessus.
- Dépassement des capacités matérielles de la cible, en cas de plusieurs versions de hardwares, notamment les capacités des mémoires (RAM ou de stockage).
- Compatibilité des données en mémoire stockée entre les différentes versions logicielles. Privilégier des formats de données extensibles : TLV, XML, JSON, YAML.
- Conservation du logiciel précédent en double zone mémoire, pour un mécanisme de récupération, ou *failsafe*, en dernier recours en cas d'erreur non gérée.

5. Assurance qualité

Comme évoqué en introduction (cf. §1.3), l'environnement de développement contribue à la qualité du logiciel développé.

Revenons finalement sur quelques principes généraux permettant d'assurer cette qualité tout au long du développement.

5.1. Versioning / Gestion de configuration

Un des premiers piliers de l'assurance qualité dans le développement logiciel est la gestion de configuration (*configuration tracking* ou *versioning*).

5.1.1. SemVer

Pour commencer, il est utile de savoir nommer ses versions logicielles efficacement.

Pour ce faire, SemVer vient aujourd'hui comme un standard en la matière : <https://semver.org/>.

SemVer, pour *Semantic Versioning*, préconise le nommage des versions avec 3 indices **MAJOR** . **MINOR** . **PATCH** :

1. **MAJOR** :

- Un changement d'indice majeur dénote d'une rupture d'API.
- L'indice MAJOR 0 (0.x.y) correspond aux développements initiaux. La stabilité de l'API n'est pas garantie tant qu'on n'a pas publié une version 1.0.0.

2. **MINOR** :

- Ajout de fonctionnalités sans rupture d'API.

3. **PATCH** :

- Corrections de bugs sans rupture d'API.

On utilise souvent le verbe *bump* en Anglais lorsqu'on incrémente l'un des indices ci-dessus.

Le concept clé reste la question de la rupture d'API. C'est ce qui permet notamment de faciliter les calculs de dépendances de packages (cf. §2.3).

5.1.2. SCM - git

Pour le suivi des modifications du code source, un outil de SCM (*Source Control Management* ou *Source Code Management*) est un socle indispensable.

Après CVS (1990), puis Subversion (2000, Apache), git (2005, Linus Torvalds, auteur du kernel Linux) s'est imposé dans les années 2010 comme l'option de référence jusqu'à aujourd'hui.

5.1.2.1. Git, les bases

Parmi les intérêts premiers de git, on identifie :

- **des capacités d'historisation** : Capacité à tracer l'historique des modifications, chaque *commit* représentant un jeu de modifications sur les fichiers du *dépôt*, donnant la possibilité éventuellement de revenir en arrière au besoin.
- **des auteurs et des commit logs** : Informations associées à chaque commit. Donne des éléments d'explication sur l'objet des modifications apportées.
- **des branches & des merges** : Permet le travail collaboratif.

Pour découvrir les fonctionnalités de base de git, parcourir le TP proposé ci-après.

👤 ✨ TP : [Git, les bases](#) (TODO : à finaliser)

5.1.2.2. Investiguer avec git

L'intérêt d'un SCM tel que git est de savoir suivre l'historique du code source de manière fine.

On peut même reposer sur cette gestion d'historique pour nous aider à investiguer sur des problèmes.

👤 TP : Investiguer avec git

5.1.2.3. Stratégie git

Une des premières choses à décider au moment de démarrer un projet avec git est la stratégie de branches et de merges qu'on va suivre.

5.1.2.3.1. Format des commit logs

La qualité et l'homogénéité des commit logs est importante pour la maintenabilité.

C'est d'autant plus vrai lorsqu'on livre du code source à un client, auquel cas l'historique git peut être considéré comme un livrable à part entière.

👤 TP : Good git commit log (TODO)

Mémo :

- <https://www.freecodecamp.org/news/how-to-write-better-git-commit-messages/>
- <https://www.conventionalcommits.org/en/v1.0.0/>
- <https://opencommits.org/>
- Ref ticket associé, cf. §5.1.3.2

5.1.2.3.2. Commit initial

Pas vraiment un élément de stratégie, mais un point de détail au moment de démarrer l'historique git : on voudra certainement partir sur une branche de développement, laquelle devra être tirée d'un commit initial.

Quoi mettre donc dans ce commit initial ?

Git ne permettant pas de créer un commit vide par défaut, une solution de contournement peut être d'ajouter un fichier `README.md` vide, à compléter par une première branche de développement livrée.

Ou probablement mieux : on pourra utiliser l'option `--allow-empty` pour forcer la création d'un premier commit vide.

5.1.2.3.3. Merge v/s rebase

Avant toute chose, il convient d'expliquer la différence entre un *merge* et un *rebase* sous git.

Un **merge** entre deux branches ayant divergé constitue à créer un nouveau noeud de fusion entre les deux branches concernées.

Dans la mesure du possible, git fusionne les deux historiques automatiquement. Dans le cas contraire, les conflits de fusion doivent être résolus avant de pouvoir enregistrer le commit de merge.

i Fast-forward

Lorsque la branche mergée est simplement en retard par rapport à la branche cible, git propose un *fast-forward*, c'est-à-dire un réaligement trivial en faisant glisser la branche mergée jusqu'à la branche cible, sans ajouter de commit de merge.

Ce comportement par défaut peut être interdit à l'aide d'une option `--no-ff`. Il peut être vérifié / assuré à l'aide d'une option `--ff-only`.

⚠ Merges et suppression de branches

Si on a mergé une branche de développement dans la branche principale, la branche de développement ne pourra plus être supprimée, car le nouveau commit de la branche principale devient dépendant des commits de la branche de développement mergée.

Un **rebase** entre deux branches ayant divergé consiste à déplacer les noeuds de la branche rebasée à la suite de la branche cible.

Cela est équivalent à faire des *cherry-picks* (i.e. applications de patches) successifs des commits de la branche rebasée sur la branche cible, puis la branche rebasée est supprimée.

Pour chaque commit rebasé, si conflits de fusion, ceux-ci doivent être résolus avant de pouvoir rebaser le commit suivant.

⚠ **Rebases et modification d'historique**

Il faut avoir conscience qu'un *rebase* implique un déplacement des commits et donc une modification d'historique.

Pour réaligner une branche de développement avec la branche principale, il est généralement préférable de privilégier un *merge* à un *rebase*, de sorte à éviter ce déplacement de commits, et donc la perte d'un niveau de qualification associé, pour pouvoir faire des *git bisect* notamment (cf. §5.1.2.2).

5.1.2.3.4. Stratégie de branching

On identifie plusieurs stratégies de branching dans la littérature sur Internet :

- **Trunk-Based Development :**
 - Tout le monde travaille sur une seule branche.
 - Induit des *rebases* fréquents.
 - Pros :
 - Limite les risques de divergences et donc de merges conflictuels.
 - Stratégie privilégiée en CI/CD.
 - Cons :
 - Permet plus difficilement de qualifier des états stables et livrables sur la branche principale.
- **GitFlow :**
 - Cf. <https://nvie.com/posts/a-successful-git-branching-model/>
- **GitHub Flow :**
 - Cf. <https://docs.github.com/en/get-started/using-github/github-flow>
- **GitLab Flow :**
 - Cf. <https://about.gitlab.com/fr-fr/topics/version-control/what-is-gitlab-flow/>

N'ayant pas encore pris le temps d'étudier dans le détail les différentes options GitFlow, GitHub Flow ou GitLab Flow, je présente à défaut une stratégie de branching que j'ai déjà pratiquée et qui a fait ses preuves à mon sens :

- Une branche de développement par feature développée.
 - Réalignements de la branche principale vers les branches de développement en *merges* standard.
- Avant de livrer :
 - Réalignement final de la branche principale vers la branche de développement avec un *merge* standard.
 - Déroulement des vérifications : compilation, qualimétrie, tests de non-régression, ...
 - Stratégie de pull-request en *merge* avec les options suivantes :
 - *--ff-only* pour s'assurer que la branche de développement est bien alignée la dernière version de la branche principale, et donc que les vérifications réalisées sont viables pour l'état courant de la branche principale.
 - *--squash* pour créer un unique noeud de synthèse pour la fonctionnalité mergée sur la branche principale, sans dépendance avec la branche de développement.
 - Faire l'effort de synthétiser le commit log pour le commit de merge.
- Au moment opportun, les branches de développement peuvent être supprimées.

On ne garde ainsi que le fil de la branche principale, avec les livraisons successives des différentes fonctionnalités.

5.1.2.3.5. Tags

Rien de très compliqué pour les tags.

On reprendra généralement le numéro de la version SemVer (cf. §5.1.1) :

- soit avec un "v" minuscule devant - pratique répandue, probablement historique (1),
- soit avec un "V" majuscule devant - moins commun,
- soit tel quel (1).

L'essentiel est de rester homogène.

Exemples :

- Le projet Scapy préfixe ses tags avec un "v" minuscule : <https://github.com/secdev/scapy/tags>.
- SemVer utilise un préfixe un "v" minuscule également : <https://github.com/semver/semver/tags>.
- Cargo utilise la version SemVer sans préfixe : <https://github.com/rust-lang/cargo/tags>.

? (1) Tagging sans préfixe, pratique plus actuelle ?

Quelques références intéressantes sur le sujet :

- <https://stackoverflow.com/questions/2006265/is-there-a-standard-naming-convention-for-git-tags#2011372>
- <https://semver.org/#is-v123-a-semantic-version>

Compte tenu de la clarification, puis du retrait de la spécification "vX.Y.Z" de SemVer entre les versions 1.0.0 et 2.0.0, on suppose que l'adoption d'une stratégie de tagging sans préfixe "v" est une pratique plus récente, peut-être en développement dans les nouveaux projets ? tels que les projets de l'écosystème Rust par exemple.

Pure supposition, à confirmer.

5.1.2.4. Plateformes git

Des plateformes complètent les fonctionnalités de git (programme de base) :

Plateform Git	URL	Fournisseur	Commentaires	Date
GitHub	https://github.com/	Microsoft (achat en 2018)	Plateforme Git de référence, notamment pour l'hébergement des projets Open Source. Gratuit pour un usage personnel.	2008
BitBucket (ex Stash)	https://bitbucket.org/	Atlassian	Avec Jira (bugtracker) et Confluence (documentation CMS), fait partie du socle édité par Atlassian.	2008
GitLab	https://about.gitlab.com/	GitLab	Projet Open Source démarré en 2011, offres commerciales pour les entreprises.	2011
Gitea	https://about.gitea.com/	Gitea Limited (depuis 2022)	Plateforme disponible gratuitement en version Open Source. Option intéressante pour une installation on-premise.	2016

Parmi les fonctionnalités complémentaires apportées par ces plateformes, on note :

- l'hébergement d'un dépôt git central,
- l'organisation des revues de code, au moyen de *pull-requests* ou *merge-requests*,
- l'intégration et le couplage avec un bugtracker (cf. §5.1.3.1 et §5.1.3.3),
- des fonctions d'organisation du travail et de pilotage projet (cf. §5.1.3.3),
- la mise à disposition d'un CMS (wiki ou Confluence),

💡 Astuce : Préférence format texte + git

Quitte à écrire de la documentation attachée au dépôt, préférer probablement l'usage d'un format texte suivi sous git (cf. §5.3.3).

- des fonctions de CI/CD (cf. §2.5.4.2 et §4.3.1).

💡 Astuce : éviter les traductions françaises des plateformes git

Eviter les traductions françaises sur les plateformes git, elles sont généralement atroces :

- "commit" devient "engagement",

- "pull-request" devient "demande de tirage",
- ...

Prendre le temps d'aller dans les configurations utilisateur pour privilégier la langue anglaise.

🧑🏻‍💻 TP : [GitHub](#)

🧑🏻‍💻 TP : Installation Gitea on-premise (TODO)

5.1.2.5. Git, mode expert

Pour découvrir des fonctionnalités avancées de git, parcourir le TP proposé ci-après.

🧑🏻‍💻 TP : Git, mode expert (TODO)

5.1.3. Ticketing

Un système de tickets permet également de contribuer à la maîtrise du contenu fonctionnel du logiciel.

5.1.3.1. Fonctionnement général

Les plateformes git identifiées précédemment (cf. §5.1.2.4) viennent généralement avec leur système de ticketing, souvent donné dans une vue nommée *issues*.

i Suite Atlassian

Pour la suite Atlassian, l'outil appairé à la plateforme git BitBucket se nomme Jira pour la gestion des tickets.

Les tickets permettent de tracer différents faits techniques caractérisant le fonctionnement du logiciel :

- des bugs,
- des souhaits d'évolutions.

Selon les plateformes (GitHub notamment), les pull-requests peuvent également être identifiées et référencées par des numéros de tickets.

📌 Références croisées entre tickets

Les outils de ticketing permettent de faire très simplement des références entre tickets.

Sous Jira, il suffit de donner l'identifiant de l'issue, généralement composé d'un nom de projet suivi d'un indice.

Sous GitHub, il suffit de préfixer le numéro d'issue avec le caractère #.

On obtient ainsi un lien permettant de naviguer rapidement entre des sujets corrélés.

💡 Tickets : espaces d'instruction de sujets techniques

On peut voir les tickets comme des espaces ou dossiers permettant d'instruire des sujets techniques.

Exemple :

- Un dysfonctionnement aléatoire est constaté sur une version logicielle donnée.
- On ouvre un ticket de bug permettant d'enregistrer les éléments connus à date :
 - version du logiciel,
 - contexte d'utilisation,
 - données d'entrées,
 - logs,
 - ...
- D'autres informations peuvent être consignées ultérieurement, comme :
 - la constatation du même problème sur une autre version du logiciel,
 - la constatation du même symptôme dans d'autres circonstances,

- des éléments d'analyses techniques,
- des éléments de surveillance du taux d'occurrence,
- ...
- Et ce jusqu'à :
 - identifier la cause du problème, et suivre le sujet jusqu'à la correction du problème,
 - voire déterminer qu'il s'agissait finalement d'un faux problème, et clore le sujet en connaissance de cause.

5.1.3.2. Intégration avec git

Comme introduit aux [§5.1.2.2](#) et [§5.1.2.4](#), les plateformes git réalisent une intégration bien utile entre git d'une part, et le système de ticketing d'autre part.

• De git vers les tickets

De la même manière qu'il est possible de créer des références croisées entre tickets dans l'outil de ticketing (cf. [§5.1.3.1](#)), la même syntaxe peut être utilisée dans les commit logs.

C'est d'ailleurs un standard que de donner dans l'entête du commit log la référence du ticket motivant la modification des sources (cf. [§5.1.2.3.1](#)).

Ceci permet de naviguer rapidement depuis l'historique git vers les tickets associés, donnant ainsi de l'information complémentaire sur l'objet des modifications.

• Des tickets vers git

Inversement, dans l'outil de ticketing, les tickets présentent les commits, les branches et les pull-requests associés au ticket.

Ceci permet de naviguer rapidement depuis un ticket vers les historiques git correspondants.

🧑🔧 TP : GitHub issues (TODO)

5.1.3.3. Organisation du travail d'équipe

Les tickets peuvent également servir à organiser le travail d'équipe.

Des vues type *Release plan* permettent de prioriser et planifier la prise en compte des features et corrections dans les prochaines versions du logiciel.

Feature release

Manage your team's prioritized work items when planning for a feature release 🚀

[Prioritized backlog](#)
[Status board](#)
[Roadmap](#)
[Bugs](#)
[In review](#)
[+ New view](#)

Filter by keyword or by field

Title	Status	Assignees	Size	Estimate
P0 2 Estimate: 2				
P1 4 Estimate: 6				
3 Updates to velocity of the ship and alien movements #802	In progress	Dedac	S	
4 Create user account and subscription #820	In progress	j0siepy	S	
5 Finalize upsell model #1323	In progress	marywhite	M	
6 Add a leaderboard for Europe #1378	In progress	rileybroughten	L	
+ Add item				
P2 2 Estimate: 3				
7 Save score across levels #800	To-do	Mattamorphic	M	
8 Review engagement metrics once posts are live #26	To-do	ohiosveryown	S	
+ Add item				

Feature release • GitHub

Manage your team's prioritized work items when planning for a feature release 🚀

Source : <https://github.com/>, fonction *Projects*

Des vues type *Scrum* ou *Kanban* permettent de présenter visuellement les travaux en cours. Les tickets sont présentés en colonnes, de gauche à droite, les colonnes de gauche correspondant aux travaux à réaliser, et les colonnes de droite aux travaux terminés.

Kanban

Visualize the status of your project and limit work in progress

[Backlog](#)
[Priority board](#)
[Team items](#)
[Roadmap](#)
[In review](#)
[+ New view](#)

Filter by keyword or by field

Backlog	Ready	In progress	In review
Backlog 2/5 Estimate: 3 This item hasn't been started - planning-tracking-demo #1366 Explore a bug - planning-tracking-demo #821 Updates to alien, beam, bomb and cannon sprites	Ready 1 Estimate: 1 This is ready to be picked up planning-tracking-demo #797 Easter egg with high score unlocking new paid levels	In progress 2/3 Estimate: 2 This is actively being worked on - planning-tracking-demo #1304 Make sure that it delights customers - planning-tracking-demo #829 Hero site - Copy	In review 1/5 Estimate: 1 This item is in review planning-tracking-demo #1101 [Tracking] Integrate payments system

Kanban • GitHub

Visualize the status of your project and limit work in progress

Source : <https://github.com/>, fonction *Projects*

i Granularité des tâches

De sorte à permettre le raffinement des tâches en sous-tâches réalisables dans des temps limités (typiquement un sprint SCRUM), on considère généralement la hiérarchie de tickets suivante :

- **Epic**

Macro-ticket décrivant une feature complète, mais nécessitant d'être raffiné en plusieurs User-Stories.

- **User-Story**

Ticket décrivant une fonction ou sous-fonction réalisable dans un temps limité.

- **Task**

Raffinement possible d'une User-Story.

Utile lorsque :

- le travail est parallélisable,
- le travail nécessite un suivi détaillé pour en assurer l'exhaustivité,
- ...

5.1.4. Index de configuration

! Importance de l'index de configuration

Lorsqu'il est constitué, un index de configuration devient rapidement un élément central de la conduite d'un projet.

Il permet d'assurer la maîtrise dans le temps des jeux des documents, logiciels et outils applicables pour chacune des *baselines*.

Il s'agit typiquement d'un tableau à double entrées :

1. Dans un premier axe, on recense tous les éléments constitutifs du projet : documents, logiciels produits, librairies utilisées ou COTS (*Commercial Off-The-Shelf*), outils, ... On présente une ligne par version pour chaque élément.
2. Dans un second axe, on identifie des *baselines*, i.e. des dates clés dans l'avancement du projet : jalons, revues, changements de périmètre, livraisons de versions, ...

Ensuite, par un système de croix, on fait correspondre les versions applicables de chaque élément pour chaque *baseline*.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
	Item	Version	Date	Location	License	MCR Mission Concept Review 17-01-01	SR System Requirements Review 17-02-01	MDR Mission Definition Review 17-03-01	SDR System Design Review 17-04-01	PDR Preliminary Design Review 17-05-01	CDR Critical Design Review 17-06-01	PRR Production Readiness Review 17-07-01	TRR Test Readiness Review 17-08-01	CCB#1 2025-08-10	CCB#2 2025-08-20	SAR System Acceptance Review 17-09-01	ORR Operational Readiness Review 17-10-01	FRR Flight Readiness Review 17-11-01
2	Documentation																	
3	System Requirements																	
4	SR	1.0.0	2025-01-15	/system/SR/SR-1.0.0.pdf	-		x											
5	SR	1.0.1	2025-02-02	/system/SR/SR-1.0.1.pdf	-			x										
6	SR	1.0.2	2025-03-02	/system/SR/SR-1.0.2.pdf	-				x	x	x	x	x	x	x	x		
7	System Design																	
8	SD	1.0.0	2025-03-15	/system/SD/SD-1.0.0.pdf	-				x									
9	SD	1.0.1	2025-04-02	/system/SD/SD-1.0.1.pdf	-					x								
10	SD	1.0.2	2025-05-02	/system/SD/SD-1.0.2.pdf	-						x	x	x	x	x	x		
11	System Test Plan																	
12	STP	1.0.0	2025-07-15	/system/STP/STP-1.0.0.pdf	-								x	x	x	x		
13	System Test Results																	
14	STR	1.0.0	2025-08-05	/system/STR/STR-1.0.0.pdf	-									x				
15	STR	1.0.1	2025-08-15	/system/STR/STR-1.0.1.pdf	-										x			
16	STR	1.0.2	2025-08-25	/system/STR/STR-1.0.2.pdf	-											x		
17	Software Requirements																	
18	SwR	1.0.0	2025-04-15	/sw/SwR/SwR-1.0.0.pdf	-					x	x	x	x	x	x	x		
19	Software Design																	
20	SwD	1.0.0	2025-04-15	/sw/SwD/SwD-1.0.0.pdf	-					x								
21	SwD	1.0.1	2025-05-02	/sw/SwD/SwD-1.0.1.pdf	-						x	x	x	x	x	x		
22	Software Test Plan																	
23	SwTP	1.0.0	2025-07-15	/sw/SwTP/SwTP-1.0.0.pdf	-								x	x	x	x		
24	Software Test Results																	
25	SwTR	1.0.0	2025-08-05	/sw/SwTR/SwTR-1.0.0.pdf	-									x				
26	SwTR	1.0.1	2025-08-15	/sw/SwTR/SwTR-1.0.1.pdf	-										x			
27	SwTR	1.0.2	2025-08-25	/sw/SwTR/SwTR-1.0.2.pdf	-											x		
28	Software					x	x	x	x	x	x	x	x	x	x	x	x	x
29	Software.js																	
30	Software.js	1.0.0	2025-08-05	/sw/versions/SoftwareABC-1.0.0.tgz	-									x				
31	Software.js	1.0.1	2025-08-15	/sw/versions/SoftwareABC-1.0.1.tgz	-										x			
32	Software.js	1.0.2	2025-08-25	/sw/versions/SoftwareABC-1.0.2.tgz	-											x		
33	COTS					x	x	x	x	x	x	x	x	x	x	x	x	x
34	JQuery																	
35	JQuery	3.7.1	2023-08-28	https://github.com/jquery/jquery/archive/refs/tags/3.7.1.tar.gz	MIT									x	x	x		
36	Tools					x	x	x	x	x	x	x	x	x	x	x	x	x
37	Vite.js																	
38	Vite.js	7.0.6	2025-07-24	https://github.com/vitejs/vite/archive/refs/tags/v7.0.6.tar.gz	MIT													
39	Vite.js	7.1.0	2025-08-07	https://github.com/vitejs/vite/archive/refs/tags/v7.1.0.tar.gz	MIT													
40	Vite.js	7.1.1	2025-08-08	https://github.com/vitejs/vite/archive/refs/tags/v7.1.1.tar.gz	MIT													
41	Vite.js	7.1.2	2025-08-12	https://github.com/vitejs/vite/archive/refs/tags/v7.1.2.tar.gz	MIT													
42	Vite.js	7.1.3	2025-08-19	https://github.com/vitejs/vite/archive/refs/tags/v7.1.3.tar.gz	MIT													

💡 Astuces de présentation dans un tableau

On pourra utiliser la fonction de filtres sur les colonnes de *baselines* pour retrouver rapidement la liste des éléments applicables pour une *baseline* donnée.

On pourra également utiliser la fonction de groupement des lignes pour permettre de replier / déplier toutes les versions d'un même élément.

Voir l'exemple de tableau correspondant au visuel présenté juste avant : [configuration-index.ods](#).

💡 Index de configuration & index documentaire

En ajoutant simplement une colonne donnant le lieu d'enregistrement de chaque version d'élément, un index de configuration peut faire office d'index documentaire par la même occasion.

💡 Index de configuration & conformité juridique

Penser à ajouter une colonne indiquant la licence des librairies et outils utilisés.

En faisant cela, on s'assure de se poser la question de la conformité juridique pour chaque élément concerné.

5.1.5. PLM - Product Lifecycle Management

Le déploiement d'un outil de PLM (*Product Lifecycle Management*) est une pratique plutôt issue du monde de l'industrie, automobile notamment pour ses origines.

Ces système permettent de décomposer un produit en sous-parties, elles-mêmes décomposées en sous-parties, jusqu'à l'identification des pièces unitaires.

Ils sont généralement connectés à l'ERP de l'entreprise, et permettent de :

- gérer les stocks de matières premières,
- éditer des BOMs (*Bills Of Materials*) pour passer des commandes de sous-parties entières à des sous-traitants,
- tracer les configurations des produits livrés,
- ...

Ci-après, une liste des principaux outils de PLM :

Source : <https://chat.mistral.ai/>, sous réserve de confirmation des informations.

Produit	Description	Date
Siemens Teamcenter	Adapté aux grands groupes industriels	1990's
Oracle Agile PLM	Solution complète, conformité réglementaire, intégration ERP	1990's
Windchill	Utilisé dans l'aérospatial, la défense, le médical	1998
Arena PLM	Interface conviviale, SaaS, adapté aux PME	2000
Autodesk Fusion Lifecycle	De la conception à la production, collaboration améliorée, cloud	2010's
Odoo PLM	Economique, intégration native avec les autres modules Odoo (ERP, CRM)	2015

Et le logiciel dans tout ça ?

Dans le cas des logiciels embarqués, ces derniers sont généralement référencés dans le PLM en tant qu'éléments nécessaires à la production d'un produit livrable.

Certaines entreprises enregistrent en sous-parties du logiciel les librairies et frameworks embarqués dans le logiciel, pouvant faire l'objet de contraintes de licences, Open Source et commerciales notamment (cf. §2.3.3), et particulièrement des *runtimes* (payables à l'unité).

En ce sens, un PLM peut contribuer au contrôle de la conformité commerciale et juridique pour les activités logicielles, notamment dans le cas des logiciels embarqués.

5.1.6. Archivage

Dernier rempart de la gestion de configuration : l'archivage.

L'objectif est de savoir remettre en place les activités de développement a posteriori : maintenance à long terme, ou restauration en cas de sinistre.

En matière de résilience aux sinistres (piratage, incendie, inondations, ...), on parle de PCA (Plan de Continuité d'Activité) et de PRA (Plan de Reprise d'Activité). Les archivages contribuent plutôt aux PRA.

On prendra soin d'archiver :

- les code sources,
- les binaires produits,
- ⚠ sans oublier les outils ! de sorte à pouvoir restaurer l'ensemble de l'environnement de développement.

Tester les archives

Archiver c'est bien, mais tant qu'on n'a pas testé nos archives et nos PRA, on n'est jamais certain que nos archives sont valides et se suffisent à elles-mêmes.

C'est pourquoi il est de bon ton de tester les procédures de PRA régulièrement.

Profiter de l'arrivée des nouveaux membres dans l'équipe

Profiter de l'arrivée des nouveaux pour tester ces PRA.

C'est censé être suffisamment bien documenté pour qu'ils puissent être autonomes, et c'est formateur.

C'est par ailleurs une bonne "épreuve du feu" pour le PRA, car le nouvel arrivant n'a pas de connaissances présumées sur le projet. S'il manque des précisions, voire des éléments, c'est l'occasion d'améliorer la documentation, voire de compléter les archives.

5.2. Automatisations

Comme évoqué précédemment Dès lors qu'on peut automatiser des actions répétitives, cela constitue généralement un risque en moins d'erreurs manuelles.

Pour cela, la mise en oeuvre de scripts peut remplir cet objectif (cf. §2.5.4.1).

L'utilisation d'une chaîne CI (*Continuous Integration*) permet en plus d'automatiser le déclenchement de traitements (cf. §2.5.4.2).

⚠ **Risque : Automatisation, de la magie à l'obscurantisme**

Suite à l'automatisation des tâches, attention à ne pas perdre, au fil du temps, la maîtrise sur ces tâches automatisées.

On a pu voir des situations où, en l'absence du serveur Jenkins fournissant un job donné, on n'était plus capable de réaliser l'opération à la main, car on ne savait plus en détails ce que faisait ce job, ou comment il travaillait.

Automatiser, c'est bien. Mais attention à ce que la magie ne se transforme pas en obscurantisme !

Au moment d'automatiser une tâche, penser à la maintenabilité de l'automatisation : enregistrement fiable (idéalement sous git) et documentation a minima.

Pour les développements Web, la mise en oeuvre d'une chaîne CD (*Continuous Delivery*) prolonge la CI jusqu'à la livraison en production (cf. §4.3.1), idéalement en GitOps (cf. §4.3.2).

5.3. Documentation

On a déjà vanté les mérites de la documentation notamment pour l'industrialisation de l'environnement de développement (cf. §2.5.6).

Cette section est dédiée à la gestion de la documentation : enregistrement, format, ...

⚠ **Ecrire des docs utiles**

Attention toutefois à ne pas tomber dans des excès.

Garder en tête qu'écrire de la documentation, cela implique également l'engagement de la maintenir dans le temps.

On risque sinon de se retrouver dans une situation avec un volume conséquent de documentation, dont une bonne partie obsolète, et sans savoir quelle documentation est obsolète ou pertinente.

💡 **Anticiper le caractère temporel d'une doc**

Pour limiter les risques de manque de discernement sur le caractère obsolète ou non de la documentation, on pourra avoir le réflexe de tagguer la caractère temporel de celle-ci lorsque cela s'y prête.

Cela pourra se faire simplement en spécifiant une information de date dans le titre du document par exemple.

Exemples :

- Inventaire **en date du xx/xx/xxxx**,
- Analyse technique **en date du xx/xx/xxxx**,
- ...

On pourra aussi reposer sur les tickets (cf. §5.1.3) pour attacher les informations liées à un sujet temporel donné, plutôt qu'enregistrer une doc temporaire dans l'espace général.

5.3.1. GED / répertoires partagés

Un premier support de documentation naturellement pratiqué est l'utilisation de formats type Office (Microsoft ou version libre) pour les documents : Word, Excel, ou PowerPoint principalement.

Il reste le besoin de disposer d'un espace d'enregistrement central, de sorte à ce que l'ensemble de l'équipe puisse interagir autour de ces documents :

- **Répertoires partagés**

C'est une solution assez directe, facilement accessible depuis un poste standard.

Les répertoires partagés ne gèrent pas nativement la dimension "gestion de configuration". C'est pourquoi on est régulièrement contraints de rajouter des informations de dates dans les noms de fichiers et de répertoires.

- **GED**

Les outils de GED (Gestion Electronique de Documents), tels Sharepoint par exemple, intègrent nativement une dimension de versioning des documents.

Attention toutefois à la lourdeur d'utilisation de certaines solutions, qui peut gréver l'efficacité pour la manipulation de documents.

5.3.2. CMS / Wiki

Autre solution souvent plébiscitée dans les entreprises : l'utilisation d'un CMS (*Content Management System*) ou Wiki, notamment la solution Confluence d'Atlassian.

Ce type de solution présente l'avantage de rapidement écrire de la documentation joliment présentée, facilement accessible en mode Web, avec des fonctions de recherches dans l'ensemble des pages.

Ces systèmes viennent également avec un certain nombre d'objets permettant de faire des mises en forme sympatiques.

⚠ CMS & piège de l'obsolescence des infos

De mon expérience personnelle, on arrive régulièrement à une situation d'obsolescence de l'information dans les Wikis.

En effet, avec ce côté presque ludique, on peut facilement écrire de la documentation bien présentée, faire des recherches, la référencer via des liens, ...

Mais l'engagement de maintenir la documentation n'est pas assuré.

Face à de l'information potentiellement obsolète, je crois voir qu'il existe un sentiment de manque de légitimité pour corriger la documentation : "ce n'est pas moi qui ai écrit cette doc, je ne sais pas si j'ai bien le droit de la modifier".

Dans cette situation, le réflexe est souvent de créer de nouvelles pages. La base documentaire ne fait que grossir, sans discernement évident entre les infos pertinentes ou obsolètes.

5.3.3. Formats texte + git

Dès lors qu'une documentation peut être attachée à l'existence d'un dépôt git, l'utilisation d'un format texte constitue une option sérieuse.

On bénéficie dès lors de toute la puissance de git pour le suivi de version du fichier texte : historique, contributions / merges, ...

Des formats textes permettent également de décrire des schémas, ce qui est largement meilleur que de dessiner ses schémas sous Visio ou autre format de fichier binaire, impossibles à suivre en historique dans le détail des modifications, et encore moins à merger !

5.3.3.1. Markdown

En tant que développeur logiciel, il convient de s'intéresser en premier lieu au format Markdown.

Ressources :

- Site officiel : <https://daringfireball.net/projects/markdown/>
- Autre guide dédié : <https://www.markdownguide.org/>

L'intérêt de ce format est de proposer un formalisme texte simple, déjà lisible dans sa forme brute, et permettant de spécifier des mises en forme documentaires, typiquement pour produire du HTML.

On ne peut pas tout faire avec Markdown, mais c'est souvent suffisant pour la constitution de docs techniques.

C'est par ailleurs largement diffusé dans les outils (GitHub, ...), d'où l'importance de s'y intéresser.

🧑🏻‍💻 TP : Format Markdown (TODO)

Mémo :

- Plugin VS Code utile : [Markdown PDF](#) par yzane

5.3.3.2. RST / ReStructured Text

Si les possibilités du format Markdown ne suffisent pas :

- gestion des tableaux limitée,
- impossible de splitter une documentation dans plusieurs fichiers Markdown source,
- ...

on pourra s'intéresser au format ReStructured Text.

Ressources :

- Référence du format : <https://docutils.sourceforge.io/rst.html>
- Guide par Sphinx : <https://www.sphinx-doc.org/en/master/usage/restructuredtext/index.html>

Ce format est issu du monde Python, Sphinx (<https://www.sphinx-doc.org/en/master/index.html>) étant l'outil permettant de compiler la doc, typiquement en HTML.

La plateforme readthedocs.io (<https://docs.readthedocs.com/platform/stable/index.html>) permet une intégration de Sphinx pour la publication de la documentation en ligne.

🧑🏻‍💻 TP : Format ReStructured Text (TODO)

5.3.3.3. PlantUML

PlantUML est un outil Open Source, implémenté en Java, sorti en 2009 dans sa première version.

Il propose un formalisme texte pour décrire des schémas UML :

- Sequence diagram
- Usecase diagram
- Class diagram
- Object diagram
- Activity diagram
- Component diagram
- Deployment diagram
- State diagram
- Timing diagram
- ...

Ressources :

- Site officiel : <https://plantuml.com/>
- GitHub : <https://github.com/plantuml/plantuml>

Des outils en ligne permettent de tester la syntaxe :

- <https://www.plantuml.com/plantuml/uml/>

🧑🏻‍💻 TP : Format PlantUML (TODO)

? Difficultés d'intégration

Etant écrit en Java, PlantUML est probablement moins facile à intégrer que d'autres formats tels que Mermaid présenté ci-après.

A confirmer.

J'ai eu l'occasion de l'intégrer dans la production d'une documentation RST (cf. §5.3.3.2) avec un séquençement à base de Makefile (cf. §2.4.4.1).

Cela nécessitait effectivement :

- le téléchargement et l'installation d'un jar,
- donc l'installation préalable d'une JRE,
- l'écriture de quelques règles Makefile,
- et donc l'installation préalable d'un binaire `make`.

5.3.3.4. Mermaid

Mermaid est un outil Open Source, implémenté en Typescript / Javascript, sorti en 2014 dans sa première version.

De même que PlantUML, il propose un formalisme texte pour décrire des schémas :

- Flowchart
- Sequence diagram
- Gantt chart
- Class diagram
- State diagram
- Pie chart
- Git graph
- Bar chart
- User Journey diagram
- C4 diagram
- ...

Ressources :

- Ancien site officiel : <https://mermaid.js.org/>
- Nouveau site officiel : <https://mermaid.ai/>
- GitHub : <https://github.com/mermaid-js/mermaid>

Des outils en ligne permettent de tester la syntaxe :

- <https://mermaid.live/edit>

TP : Format Mermaid (TODO)

Mémo :

- <https://mermaid.js.org/ecosystem/tutorials.html>
- <https://www.markdownlang.com/advanced/diagrams.html>
- Extension VS Code utile : [Mermaid](#) plugin officiel => meilleure visu, possibilité d'export
- Diagrammes utiles :
 - SW Technical charts:
 - Flowcharts
 - Sequence diagrams
 - Class diagrams
 - State diagrams
 - Architecture diagrams
 - Packet diagrams
 - GitGraph diagrams
 - Communication & organization:

- Mindmaps
- Gantt diagrams
- Kanban boards
- General charts:
 - Pie charts
 - XY charts
 - Quadrant charts
 - Radar diagrams
 - Treemap diagrams
- ⚠ Does not work with "Markdown PDF" extension.
 - Workaround:
 1. Export HTML.
 2. Then print to PDF.

? Facilité d'intégration

Implémenté en Typescript / Javascript, cet outil bénéficie peut-être d'un meilleur niveau d'intégration que PlantUML ?

A confirmer.

A noter l'intégration native de la syntaxe Mermaid avec un certain nombre de plateformes git : GitHub, GitLab (à confirmer si PlantUML est supporté également).

A noter l'abandon en juin 2025 du format PlantUML par draw.io (<https://www.drawio.com/>) au bénéfice de Mermaid : <https://www.drawio.com/blog/plantuml-to-mermaid>.

5.3.3.5. Slides

TODO - Slides with Markdown :

- Extension VS Code utile : [vscode-reveal](#) par evilz
- A expérimenter.

5.3.4. Documentation d'API

Les différents langages viennent généralement avec leur format de documentation d'API.

Ces formats de documentation permettent une documentation des API au plus proche du code :

- packages,
- classes,
- variables et attributs de classes,
- fonctions et méthodes,
- paramètres de fonctions et méthodes,
- valeurs de retour,
- exceptions potentiellement générées,
- ...

Le tableau ci-après identifie les formats de documentation associés à chaque langage :

Langage	Format de documentation	URLs
C/C++	Doxygen	https://www.doxygen.nl/
Java	Javadoc	https://docs.oracle.com/en/java/javase/25/docs/specs/javadoc/doc-comment-spec.html
JS / Typescript	JSDoc	https://jsdoc.app/

Langage	Format de documentation	URLs
PHP	PHPDoc	https://phpdoc.org/
Rust	rustdoc	https://doc.rust-lang.org/nightly/rustdoc/how-to-write-documentation.html
Python	Docstrings	https://peps.python.org/pep-0257/ , https://realpython.com/how-to-write-docstrings-in-python/
Go	Go Doc	https://go.dev/doc/comment

A noter que, à l'exception de Python et Go, tous ces formats de documentation ont tendance à ressembler au format Javadoc (probablement le plus ancien).

Une fois les API documentées dans le code, on peut extraire cette documentation sous forme de pages HTML ou document PDF.

🧑🏻‍💻 ✂ TP : Documentation Doxygen de code C/C++ (TODO)

🧑🏻‍💻 ✂ TP : Documentation Javadoc de code Java (TODO)

🧑🏻‍💻 ✂ TP : Documentation JSDoc de code Javascript et Typescript (TODO)

🧑🏻‍💻 ✂ TP : Documentation PHPDoc de code PHP (TODO)

🧑🏻‍💻 ✂ TP : Documentation rustdoc de code Rust (TODO)

🧑🏻‍💻 ✂ TP : Documentation Docstrings de code Python (TODO)

🧑🏻‍💻 ✂ TP : Documentation Go Doc de code Go (TODO)

6. Annexes

6.1. Licence

Cours mis à disposition sous licence CC BY-NC-SA 4.0.



[LICENSE.txt](#) :

Software development environment lesson © 2025 by Alexis Royer

<https://github.com/alxroyer/edu-sw-dev-env>

This work is licensed under Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International. To view a copy of this license, visit <https://creativecommons.org/licenses/by-nc-sa/4.0/>

6.2. Ressources

Images :

- <https://www.pexels.com/fr-fr/>
- <https://fr.freepik.com/>
- <https://pixabay.com/fr/>
- <https://dashboardicons.com/> (logos)
- <https://www.logo.wine/> (logos)